Swiftradish: Master Product Strategy, UX Architecture, and Engineering Blueprint

Document A: Product Strategy and Product Requirements Document

Central Product Thesis and Core Value Proposition

Contemporary social platforms suffer from a structural bifurcation: algorithmic broadcast networks such as TikTok, Instagram, and X optimize for passive, solitary media consumption, whereas real-time communication tools such as Discord and Snapchat trap shared experiences inside ephemeral, unsearchable chat logs. Attempting to combine every feature from these incumbents into an undifferentiated super-app guarantees cognitive overload, bloated operational costs, and rapid user churn.

Central Product Thesis: Swiftradish is a co-active social platform where interest-driven micro-communities and friend groups move seamlessly from rich media discovery to real-time coordination, collaborative remixing, and shared digital or real-world activities within a single unified surface.

Rather than treating posts as terminal broadcast artifacts and messaging as an isolated utility drawer, Swiftradish binds rich media creation directly to structured group action through "Co-Plots" and "Radish Patches." A post is not merely consumed; it acts as a live coordination node that can spawn a real-time thread, a collaborative media carousel, or a scheduled shared activity.

Three primary drivers sustain user adoption, friend invitations, and long-term retention after initial novelty subsides. First, action-oriented social media ("Co-Plots") allows every rich post or carousel to attach an interactive coordination block—such as a collaborative photo album where participants contribute slides, a live event RSVP, or an instant topic room—turning passive scrolling into shared doing. Second, low-friction creative remixing and curated identity allow users to express themselves through customizable dark-first profile themes, verifiable contribution milestones, and an expressive, non-manipulative mascot system ("Pip the Radish") that celebrates genuine co-creation rather than anxiety-inducing daily streak maintenance. Third, high-signal, algorithmically transparent feeds separate the chronological social graph ("My Patch") from topic-seeded discovery ("Fresh Harvest"), guaranteeing that creators reach their actual followers while giving users explicit controls over why content is recommended.

Intended Demographics and Co-Action Use Cases

Swiftradish is architected for digital-native creative cohorts, hobbyist collectives, urban social coordinators, and niche interest communities aged 18 to 34 who currently fragment their social lives across three or more incompatible applications—typically discovering an idea on TikTok or Pinterest, screenshotting it into a WhatsApp or Discord group, and using a third tool to organize the resulting project or meetup. Because Section 9 of India's Digital Personal Data Protection (DPDP) Act 2023 imposes stringent obligations requiring Data Fiduciaries to obtain verifiable

parental consent before processing the personal data of any individual under 18 years of age, the initial launch enforces a strict 18+ age gate, focusing on university cohorts, independent creators, design and technology communities, culinary and fitness clubs, and local cultural collectives.

Core User Segment	Primary Friction in Incumbent Apps	Swiftradish Co-Action Workflow	Primary Retention Loop
Urban Social Coordinators	Event ideas get buried in group chat scrollback; Instagram stories expire before plans solidify.	Publish a visual "Co-Plot" card with a 1-tap RSVP and embedded sub-chat; attendees append photos to the shared carousel afterward.	Post-activity collaborative media carousels that build a permanent shared memory archive.
Hobbyist & Creative Collectives	Reddit is text-heavy and anonymous; Discord lacks structured, discoverable visual portfolios.	Create a themed "Radish Patch" (topic space) combining a curated media grid with persistent real-time channels.	Weekly creative prompts where members submit visual remixes into a unified showcase thread.
Everyday Curators & Tastemakers	Algorithmic feeds suppress niche posts unless formatted as hyper-edited short-form video.	Publish rich mixed-media carousels (images, short video clips, formatted text notes, and deep links) indexed via full-text search.	High-signal bookmarks, remix chains, and non-gamified contribution badges.

Feature Prioritization Matrix and Evaluation Framework

To prevent engineering overextension and ensure a coherent initial product that a lean team utilizing AI coding agents can ship with high reliability, candidate features are evaluated using a weighted **Value-Differentiation-to-Risk Ratio (VDRR)**:
\text{VDRR} = \frac{(2 \times \text{User Value}) + (1.5 \times \text{Thesis Differentiation})}{\text{Implementation Effort} + \text{Reliability Risk} + \text{Operational Cost}}
Each parameter is scored on a 1–5 integer scale. Features with foundational necessity bypass scoring into Tier 1; high-VDRR features form Tier 2 (Initial Launch MVP); moderate-VDRR features form Tier 3 (Early Post-Launch); high-cost or complex features move to Tier 4 (Advanced); and thesis-diluting or operationally hazardous features are placed in Tier 5 (Rejected/Deferred).

Feature ID & Name	Tier	Problem Solved	Main User Journey	Value / Diff.	Effort / Cost	Abuse, Privacy & Safety Implications	Analytical Basis for Tier Assignment
F-01: Auth, Sessions & Profile Identity	1 (Foundation)	Secure account access and persistent	Sign up via OAuth/Email \rightarrow	5 / 2	2 / 1	Credential stuffing, handle squatting,	Mandatory security and identity layer for all

Feature ID & Name	Tier	Problem Solved	Main User Journey	Value / Diff.	Effort / Cost	Abuse, Privacy & Safety Implications	Analytical Basis for Tier Assignment
		social identity.	verify age (18+) \rightarrow claim @handle \rightarrow customize avatar, bio, and theme token.			account enumeration. Mitigated via Supabase Auth + rate limits.	authenticated actions.
F-02: Hybrid Social Graph & Privacy Controls	1 (Foundation)	Asymmetric following for discovery plus mutual trust for coordination.	Follow public creator or send follow request to private account; mutual follows auto-unlock direct co-plotting.	5 / 3	2 / 1	Stalking, unwanted contact, graph scraping. Mitigated via strict row-level/service-layer visibility checks.	Asymmetric follows drive discovery; mutual-follow detection powers safe messaging defaults.
F-03: Block, Mute, Report & Statutory Grievance Flow	1 (Foundation)	Protects users from harassment and fulfills statutory safe-harbor rules.	Tap overflow menu on post/user/message \rightarrow select Block, Mute, or Report \rightarrow receive ticket ID and 24h SLA tracking.	5 / 2	3 / 2	Weaponized false reporting, unlawful content, impersonation. Requires 24h/72h moderation queue.	Legally required under India IT Rules 2021 and essential for baseline community trust.
F-04: Rich Mixed-Media Composer	2 (MVP Launch)	Static single-image or plain-text	Open composer \rightarrow attach up to	5 / 4	3 / 2	Malicious file uploads, EXIF GPS	Core expression primitive; direct-to-R2

Feature ID & Name	Tier	Problem Solved	Main User Journey	Value / Diff.	Effort / Cost	Abuse, Privacy & Safety Implications	Analytical Basis for Tier Assignment
& Carousels		posts fail to capture multi-faceted experiences.	10 images/clips \rightarrow crop/reorder \rightarrow add formatted text, topic tags, and alt-text \rightarrow publish.			leakage, copyright infringement. Mitigated via magic-byte checks and EXIF stripping.	multipart uploads keep egress costs at $0.
F-05: Collaborative "Co-Plot" Cards & Shared Carousels	2 (MVP Launch)	Turns passive posts into shared group activities and collective memory albums.	Toggle "Make Co-Plot" in composer \rightarrow set activity time/prompt \rightarrow friends tap "Join Plot" and append media slides.	5 / 5	3 / 2	Unauthorized spam injection into public carousels. Mitigated via creator approval queue or mutual-only contributor locks.	Primary product differentiator linking feed consumption directly to doing things together.
F-06: Dual-Mode Home Feed (Chronological + Transparent Discovery)	2 (MVP Launch)	Eliminates algorithmic anxiety while solving the cold-start discovery problem.	Switch between "My Patch" (strict chronological following + joined plots) and "Fresh Harvest" (time-decayed topic/graph	5 / 4	3 / 2	Engagement-bait farming and echo chambers. Mitigated via non-addictive pagination and "Why this post?" explainabilit	Preserves user well-being and creator fairness without needing an opaque ML cluster at launch.

Feature ID & Name	Tier	Problem Solved	Main User Journey	Value / Diff.	Effort / Cost	Abuse, Privacy & Safety Implications	Analytical Basis for Tier Assignment
			score).			y badges.	
F-07: Contextual Reactions, Threaded Comments & Remixes	2 (MVP Launch)	Generic likes lack nuance; flat comments break group discussions.	React with custom radish expressions \rightarrow write nested comment (max 3 levels) or quote-remix post into a new Co-Plot.	4 / 4	2 / 1	Brigading, comment spam, XSS injection. Mitigated via strict input sanitization and per-user rate limits.	Drives the primary feedback loop for creators and sparks group conversations.
F-08: Real-Time Direct & Group Messaging (WebSockets)	2 (MVP Launch)	Coordination requires instant, reliable 1:1 and small-group communication.	Click "Open Plot Chat" on a post or start a DM \rightarrow exchange real-time text/media with delivery/read states and typing indicators.	5 / 4	4 / 2	Unsolicited DM spam, phishing links, connection exhaustion. Mitigated via message request inbox and WS connection limits.	Essential for the "doing things together" thesis; integrated tightly with posts and Co-Plots.
F-09: Unified Full-Text & Fuzzy Search + Topic Spaces ("Patches")	2 (MVP Launch)	Users must find people, niche communities, and past conversations effortlessly.	Type query in command bar \rightarrow view instant matches across @users, #topics, Patches, and posts	4 / 3	2 / 1	Search poisoning, hashtag hijacking, ReDoS queries. Mitigated via websearch_to_tsquery and trigram thresholds.	High utility at zero extra infrastructure cost using PostgreSQL tsvector and pg_trgm.

Feature ID & Name	Tier	Problem Solved	Main User Journey	Value / Diff.	Effort / Cost	Abuse, Privacy & Safety Implications	Analytical Basis for Tier Assignment
			via Postgres FTS + trigrams.				
F-10: In-App Activity Center & Web Push Notifications	2 (MVP Launch)	Re-engages users around real social coordination without dark-pattern spam.	Receive grouped in-app toast or PWA Web Push when a friend replies or a Co-Plot starts \rightarrow deep-link directly to context.	4 / 2	3 / 1	Notification fatigue, duplicate push storms. Mitigated via deduplication keys, tag management, and quiet hours.	Crucial for retention in a PWA; respects iOS 16.4+ Home Screen installation rules.
F-11: "Pip the Radish" Mascot Core Set & Non-Manipulative Badges	2 (MVP Launch)	Gives the product unmistakable warmth, visual identity, and milestone recognition.	Encounter Pip in onboarding, empty states, and upload celebrations; earn permanent profile badges for hosting Co-Plots.	4 / 5	2 / 1	Badge farming via automated spam posts. Mitigated by tying badges to verified mutual participation rather than raw volume.	Establishes brand affinity immediately; ships 30 core vector/Lottie poses at MVP, expanding to 216 later.
F-12: Short-Form Video (Up to 60s) with Managed HLS Streaming	3 (Post-Launch)	Dynamic activities benefit from short vertical or square video clips.	Select video in composer \rightarrow client validates duration/size \rightarrow direct	4 / 3	3 / 3	High bandwidth/transcoding abuse, copyrighted video uploads. Mitigated via strict daily	Deferred to Phase 12 so image/carousel stability is proven before incurring video

Feature ID & Name	Tier	Problem Solved	Main User Journey	Value / Diff.	Effort / Cost	Abuse, Privacy & Safety Implications	Analytical Basis for Tier Assignment
			upload to Cloudflare Stream \rightarrow adaptive HLS playback.			upload quotas and duration caps.	transcoding costs.
F-13: 24-Hour Ephemeral "Sprouts" (Status Updates)	3 (Post-Launch)	Low-stakes availability signaling ("Free for coffee at 5 PM").	Post a lightweight text/photo status ring on profile that auto-archives after 24 hours and allows 1-tap DM replies.	3 / 3	2 / 2	Ephemeral harassment evading moderation. Mitigated by retaining soft-deleted audit records for 30 days for safety review.	Useful for real-time coordination, but counterproductive during cold start when daily active density is initially low.
F-14: Full 216-Pose Mascot Library & Profile Theme Studio	4 (Advanced)	Deepens cosmetic personalization and emotional expressiveness across all edge states.	Unlock seasonal/contextual Pip reaction packs and custom CSS-variable profile palettes.	3 / 4	3 / 1	Visual clutter or contrast degradation. Mitigated via strict WCAG AA token validation across custom themes.	Expands the initial 30-pose core matrix once core product loops are validated.
F-15: Creator Tipping, Subscriptions & Paid Event Tickets	5 (Deferred)	Enables community organizers and creators to fund real-world or digital gatherings.	Attach paid ticket or supporter tier to a Patch or Co-Plot.	3 / 2	5 / 5	Payment fraud, chargebacks, GST/tax withholding compliance, KYC/AML burdens, refund disputes.	Explicitly deferred from MVP. Introducing financial ledgers before product-market fit creates massive

Feature ID & Name	Tier	Problem Solved	Main User Journey	Value / Diff.	Effort / Cost	Abuse, Privacy & Safety Implications	Analytical Basis for Tier Assignment
							operational drag.
F-16: Long-Form Video (>60s), Live Audio/Video Calling & E2EE	5 (Rejected)	Broadcast streaming, WebRTC calls, and cryptographic messaging.	N/A	2 / 1	5 / 5	Extreme egress/TURN server costs; E2EE breaks server-side search, multi-device web history, and statutory moderation.	Rejected for initial roadmap. E2EE conflicts with web-first searchability and lean moderation requirements.

Core Candidate Feature Areas and Staged Release Boundaries

Identity and Social Graph

Swiftradish implements a hybrid asymmetric-plus-mutual social graph. Users can follow public accounts asymmetrically to populate their feed and discover creators, or toggle their account to Private, requiring explicit follow approval. When two users follow each other, the database automatically computes a mutual relationship edge via a relational trigger. Mutual edges unlock frictionless coordination privileges—such as contributing slides to a friend's Co-Plot carousel or initiating direct chats without passing through the "Message Requests" quarantine queue. Profile verification avoids pay-to-win badges; instead, Swiftradish uses cryptographic domain verification (linking a personal domain or DNS TXT record) alongside earned community contribution badges.

Content Creation

The MVP composer supports rich Markdown-lite text (bold, italic, inline code, blockquotes, lists, and validated external links), single high-resolution images, and multi-slide carousels containing up to 10 items. Crucially, every post can be upgraded into a **Co-Plot** by attaching one of three interactive coordination modules. First, a Shared Memory Carousel allows invited users or mutuals to upload and append their own photos to the post's carousel, subject to creator moderation. Second, an Activity RSVP and Time Slot displays a structured timestamp, an optional venue or link label, a capacity limit, and a 1-tap attendance roster ("Going," "Maybe," or "Declined") that automatically provisions a dedicated group chat thread for attendees. Third, a Visual Remix Prompt allows community members to attach visual response cards in a horizontal showcase strip beneath the parent post. Drafts are persisted locally in IndexedDB and synced

to the backend, ensuring zero data loss if a browser tab closes. Mandatory alt-text prompts and automated EXIF metadata stripping protect accessibility and physical privacy.

Feed and Interaction

Swiftradish rejects opaque, engagement-maximizing black-box algorithms that optimize for outrage or compulsive scrolling. The home screen provides two explicit, user-controlled views. The default view, **My Patch**, is a 100% deterministic, reverse-chronological feed of posts from followed accounts, joined Radish Patches, and active Co-Plots, complete with a "Caught Up" divider marking where previously viewed posts begin. The secondary view, **Fresh Harvest**, is a transparent, relevance-ranked discovery feed calculated using a deterministic scoring formula: $$\text{Score} = \frac{(W_r \cdot \text{Reactions}) + (W_c \cdot \text{Unique Commenters}) + (W_p \cdot \text{Co-Plot Joins}) + (W_t \cdot \text{Topic Affinity})}{(\text{Age in Hours} + 2)^{1.6}}$$
Every card in "Fresh Harvest" includes a subtle provenance pill badge explaining why it appeared (such as *"Because you follow #analog-photography"* or *"Popular among people you follow"*), accompanied by a 1-tap control to mute the topic or author.

Advanced Messaging

Messaging is architected as a primary product subsystem rather than an isolated profile widget. In the MVP release, Swiftradish supports one-to-one conversations, group chats (up to 50 participants), and Co-Plot linked threads. Supported MVP capabilities include cursor-based message history pagination, real-time delivery and read receipts, typing indicators throttled to one event per three seconds, inline message replies, custom reactions, message editing within a 15-minute window with an explicit "edited" audit label, soft deletion, safe image attachment sharing, conversation muting and pinning, and per-conversation blocking and reporting. Voice calling, video rooms, arbitrary binary document uploads, and end-to-end encryption (E2EE) are explicitly excluded from the MVP to contain WebRTC TURN server costs, prevent malware distribution, and preserve server-side full-text message search and statutory abuse moderation.

Discovery, Search, and Communities

Public discovery centers on **Radish Patches**—lightweight, topic-anchored community spaces that bridge a curated visual feed with a persistent group chat lounge. Unlike sprawling Discord servers with dozens of inactive channels, each Patch is structurally constrained to a single curated media board, an active Co-Plots calendar, and at most three real-time chat channels. Search is powered natively by PostgreSQL combining tsvector full-text indexing with pg_trgm trigram similarity, providing instant typeahead and weighted results across usernames, display names, post bodies, hashtags, and Patches without the operational overhead of an external Elasticsearch cluster.

Notifications, Rewards, Badges, and Mascot

Notifications are bifurcated into high-priority Actionable Co-Plot Updates (RSVP changes, plot reminders, direct mentions, and direct messages) and low-priority Ambient Social Feedback (reactions, new followers, and general Patch posts). Ambient notifications are rolled up and deduplicated inside the in-app Activity Center, whereas only high-priority events trigger Web

Push notifications by default. Similarly, rather than relying on manipulative daily login streaks, Swiftradish implements qualitative contribution milestones tied to the mascot, **Pip the Radish**. Badges such as *First Sprout* (completing onboarding and joining a Co-Plot), *Plot Architect* (hosting five Co-Plots with at least three participating mutuals), *Patch Gardener* (contributing accessible posts with alt-text bookmarked by non-followers across four distinct weeks), and *Remix Catalyst* (inspiring ten unique community remixes) reward genuine social participation. Anti-farming rules enforce cooldown windows, require interactions from distinct accounts with verified age and trust scores, and immediately revoke progress if content is removed for policy violations.

Product Success Metrics and Validation Hypotheses

Product health is evaluated across three primary metrics rather than raw pageviews. The primary North Star metric is the **Co-Action Conversion Rate**, defined as the percentage of daily active users (DAU) who actively join a Co-Plot, contribute a slide to a shared carousel, or participate in a Patch thread, targeting at least 28% of DAU. Secondary metrics are **D7 and D30 Cohort Retention** (targeting \ge 35\% at Day 7 and \ge 18\% at Day 30) and **Creator Feedback Velocity**, measuring the percentage of published posts and Co-Plots that receive at least one meaningful interaction within four hours of creation (targeting \ge 65\%).

Document B: Competitive Landscape and Technical Research

Comparative Analysis of Social Architectures

To ensure Swiftradish avoids imitating structural flaws in existing platforms, seven highly relevant competitors were analyzed across interaction loops, information architecture, technical lessons, and user sentiment.

Product	Core Value Proposition & Loop	Feed, Creation & Messaging Mechanics	Public Engineering & UX Lessons	User Praise vs. Documented Criticisms	Swiftradish Differentiation Opportunity
Instagram	Visual lifestyle broadcasting and aspirational graph maintenance. Loop: Post Reel/Carousel \rightarrow harvest likes/views \rightarrow reply to Stories in DMs.	Algorithmic main feed heavily weighted toward Reels; 24h Stories bar at top; isolated DM inbox where actual friend sharing occurs via forwarded Reels.	Shifted social graph engagement into private DMs and Stories; multi-image carousels remain the highest-retention format for everyday users.	**Praise:** Polished media editing and carousel layouts. **Criticism:** Algorithmic suppression of friends' photos, ad saturation, and performative anxiety.	Elevate collaborative carousels and friend coordination to the primary surface rather than burying friend interaction inside forwarded reel DMs.
Discord	Persistent,	No unified	High efficiency	**Praise:** Fast	Combine a

Product	Core Value Proposition & Loop	Feed, Creation & Messaging Mechanics	Public Engineering & UX Lessons	User Praise vs. Documented Criticisms	Swiftradish Differentiation Opportunity
	low-latency community voice and text chat organized into hierarchical servers. Loop: Join server \rightarrow lurk or chat in text/voice channels \rightarrow react with custom emojis.	home feed; purely channel-based chronological chat streams; limited media gallery browsing.	of WebSocket gateway multiplexing and optimistic message rendering; however, chat scrollback is a poor medium for discovering evergreen visual content.	real-time presence, low friction for tight-knit groups. **Criticism:** Overwhelming channel sprawl, unsearchable history, poor asynchronous media curation.	clean visual media grid with constrained (max 3) real-time channels per "Radish Patch" so asynchronous posts do not vanish in chat noise.
Reddit	Topic-centric asynchronous forums governed by community upvotes and volunteer moderators. Loop: Browse subreddit/home feed \rightarrow read nested comment trees \rightarrow upvote/comment.	Vote-ranked (Hot/Top) or New feeds; strong nested markdown threads; weak, bolted-on real-time chat that users largely ignore.	Nested comment trees (capped depth) provide superior discourse structure compared to flat comment lists.	**Praise:** High-signal topic discovery and searchable archive. **Criticism:** Hostile anonymity, dated visual media presentation, weak social graph/friend identity.	Retain topic-first discovery and structured 3-level nested comments, but anchor identity in rich visual profiles and friend co-action.
Pinterest	Solitary visual discovery and future-intent bookmarking ("Boards"). Loop: Search/browse masonry grid \rightarrow save Pin to Board \rightarrow click external link.	Interest-graph masonry grid; strong visual search; minimal social interaction or real-time conversation.	Masonry and adaptive aspect-ratio grids maximize visual density without cropping diverse media types.	**Praise:** Calm, non-toxic interface focused on inspiration. **Criticism:** Dead social layer, proliferation of AI-generated spam images, zero group coordination.	Turn static inspiration boards into live "Co-Plots" where friends go from saving an idea to actually executing it together.
Strava /	Activity-logged	Chronological	Social networks	**Praise:**	Generalize the

Product	Core Value Proposition & Loop	Feed, Creation & Messaging Mechanics	Public Engineering & UX Lessons	User Praise vs. Documented Criticisms	Swiftradish Differentiation Opportunity
Letterboxd	social networks anchored around a specific real-world verb (running/cycling or watching films). Loop: Do activity \rightarrow log structured artifact \rightarrow give Kudos/read reviews.	activity feed enriched with structured metadata (maps/splits or star ratings/lists).	built around *doing an activity* generate dramatically higher authenticity and lower toxicity than pure status networks.	High-trust community, structured metadata cards, meaningful milestones. **Criticism:** Narrow single-vertical focus limits broader creative sharing.	"structured activity card" into flexible Co-Plots supporting any creative, local, or digital group activity.
BeReal / Locket	Ephemeral authenticity among close friends via daily prompts or home-screen widgets. Loop: Notification trigger \rightarrow capture dual-camera photo \rightarrow view friends' photos.	Single daily batch feed or widget stream; zero public discovery or searchable archives.	Novelty notification mechanics drive explosive early growth but suffer severe long-term churn once the daily prompt feels repetitive.	**Praise:** Anti-performative intimacy. **Criticism:** Repetitive loop, nothing to do inside the app once daily photos are viewed.	Reject forced daily timers; instead, drive authenticity through shared multi-author carousels around real events and projects.
Geneva	Group coordination and local club homebases combining chat rooms, event calendars, and media posts.	Room-based navigation with dedicated event RSVP cards and threaded discussions.	Proved strong demand for local/interest club coordination, but suffered from heavy mobile-only onboarding friction and lack of a cross-community discovery feed.	**Praise:** Clean integration of events and chat. **Criticism:** Siloed groups with no cross-pollination or individual creator feed.	Unify the personal social feed ("My Patch"), public discovery ("Fresh Harvest"), and community spaces ("Patches") into a single installable PWA.

Realtime Transport and Infrastructure Trade-offs

A rigorous architectural evaluation was conducted across four candidate realtime transport protocols for Swiftradish's messaging, presence, and live notification subsystems. Next.js Route Handlers operating in serverless environments cannot hold persistent WebSocket connections because connections terminate upon response completion or function timeout. Consequently, holding stateful sockets in dedicated FastAPI/Uvicorn workers backed by Redis Pub/Sub provides the optimal balance of latency, cost, and architectural control.

Transport Protocol	Connection & Scaling Characteristics	Ordering & Delivery Semantics	Failure Handling & Reconnection	Infrastructure Cost & Complexity	Recommended Swiftradish Use Case
WebSockets (FastAPI Native + Redis Pub/Sub)	Persistent full-duplex TCP connection upgraded via HTTP 101. Requires stateful ASGI server workers (Uvicorn) and Redis Pub/Sub for cross-worker fanout.	Strict FIFO ordering per TCP socket; supports application-level ACKs (client_msg_id \rightarrow server_msg_id) for at-least-once delivery with idempotent deduplication.	Requires client heartbeat (ping/pong every 25s), exponential backoff with jitter on disconnect, and cursor-based gap sync (after_id) upon reconnect.	Low direct cost on dedicated/container compute (e.g., Fly.io/Railway); incompatible with pure serverless functions that terminate connections after timeout.	**Selected for Active Chat & Presence:** Optimal for bi-directional low-latency DMs, group chats, typing indicators, and Co-Plot live rooms.
Server-Sent Events (SSE)	Unidirectional HTTP/1.1 or HTTP/2 persistent stream from server to client; client sends mutations via standard REST POST.	Ordered stream per connection; native browser EventSource supports automatic reconnection via Last-Event-ID header.	Simpler recovery than WebSockets for read-only streams, but high overhead if client frequently emits typing/presence events via separate HTTP requests.	Moderate; still holds open connections on the ASGI backend.	**Rejected as Primary Transport:** Since chat requires bi-directional typing/presence heartbeats, running both SSE and WebSockets duplicates connection state.
Short/Long HTTP Polling	Periodic REST GET /api/v1/notifications/unread requests at fixed or adaptive intervals (e.g.,	Stateless request-response; relies entirely on database/cache state.	Inherently resilient to dropped connections; zero persistent socket state required.	Zero idle connection cost, but high database/API request overhead if intervals are too short.	**Selected for Fallback & Background Sync:** Used only when WebSocket upgrades are blocked by

Transport Protocol	Connection & Scaling Characteristics	Ordering & Delivery Semantics	Failure Handling & Reconnection	Infrastructure Cost & Complexity	Recommended Swiftradish Use Case
	every 30–60s when backgrounded).				restrictive corporate proxies or for low-frequency feed "new posts available" checks.
Managed Realtime (Ably / Pusher / Supabase Realtime)	Offloads connection termination and global edge fanout to a third-party SaaS.	High reliability, built-in channel presence and history replay.	Handled by vendor SDKs.	High variable cost at scale (billed per peak concurrent connections and millions of messages, where every typing event or presence ping multiplies message bills).	**Deferred:** Kept as a fallback migration path if concurrent connections exceed 50,000+ and self-managed Redis Pub/Sub cluster operations become a bottleneck.

Evidence Register and Confidence Assessment

Source ID	Source Publication / Documentation	Date / Version	Reliability & Nature of Evidence	Key Architectural or Product Decision Supported
SR-01	Google Stitch Official Docs & Codelabs (stitch.withgoogle.com, MCP Setup & Antigravity Codelab)	Q2–Q3 2026	Primary official documentation & developer guides (High Confidence).	Confirms Google Stitch MCP server is hosted remotely at https://stitch.googleapis.com/mcp using X-Goog-Api-Key, offering 350 free generations/month and DESIGN.md token extraction.
SR-02	Cloud Object Storage Benchmarks (Cloudflare R2 vs AWS S3 vs Backblaze B2)	Apr–Sep 2026	Verified published vendor pricing & FinOps benchmarks (High Confidence).	Selects Cloudflare R2 ($0.015/GB/mo, $0 egress, 10GB free tier) + Cloudflare CDN for public

Source ID	Source Publication / Documentation	Date / Version	Reliability & Nature of Evidence	Key Architectural or Product Decision Supported
				media delivery, avoiding S3's $0.09/GB egress penalty.
SR-03	Managed Video Streaming Benchmarks (Cloudflare Stream)	2026	Published vendor pricing (High Confidence).	Selects Cloudflare Stream ($5/1,000 mins stored + $1/1,000 mins delivered, $0 egress) for Phase 12 short-form video over self-managed FFmpeg clusters.
SR-04	India IT (Intermediary Guidelines and Digital Media Ethics Code) Rules, 2021 & GAC Directives	2021–2026 Amendments	Primary statutory/regulatory framework via [span_130](start_span)[span_130](end_span)gac.gov.in (High Confidence).	Mandates in-app Grievance Officer reporting workflow with 24h acknowledgment, 24h takedown for severe bodily privacy/impersonation violations, 72h law enforcement SLA, and 15-day resolution.
SR-05	India Digital Personal Data Protection (DPDP) Act, 2023 (Section 9)	2023–2026	Primary statutory law & legal analysis (High Confidence).	Justifies enforcing an 18+ age gate at MVP launch to avoid the complex identity-verification infrastructure required for verifiable parental consent of minors under 18.
SR-06	FastAPI + Supabase Auth JWT & JWKS Integration Patterns	2024–2026	Technical implementation guides & SDK docs (High Confidence).	Establishes local cryptographic JWT verification in FastAPI via JWKS/HS256 rather than making synchronous

Source ID	Source Publication / Documentation	Date / Version	Reliability & Nature of Evidence	Key Architectural or Product Decision Supported
				network calls to supabase.auth.get_user() on every API request.
SR-07	Apple Developer Web Push Documentation & iOS 16.4–19 PWA Benchmarks	2025–2026	Official Apple docs & empirical browser testing (High Confidence).	Confirms iOS Web Push requires user-initiated "Add to Home Screen" installation, forbids silent background push, and requires explicit user gesture before permission prompts.
SR-08	Python Async Task Queue Comparisons (ARQ vs Celery)	2025–2026	Engineering benchmarks & library specifications (High Confidence).	Selects arq over Celery for FastAPI background jobs due to native asyncio execution, low memory footprint, pessimistic job timeouts, and shared Redis broker simplicity.
SR-09	PostgreSQL Official Documentation (pg_trgm & Full-Text Search tsvector)	Current (2026)	Official PostgreSQL documentation & query benchmarks (High Confidence).	Validates hybrid tsvector + pg_trgm GIN indexing with weighted ranking (ts_rank * 2.0 + similarity * 1.0) for unified search without Elasticsearch.
SR-10	Next.js App Router Backend-for-Frontend & WebSocket Guides; Starlette 1.0 / FastAPI 0.137.0 Release Notes	Mar–Aug 2026	Official Next.js docs & framework release specifications (High Confidence).	Confirms Next.js serverless routes cannot hold persistent WebSockets; mandates lifting client WebSocket state to

Source ID	Source Publication / Documentation	Date / Version	Reliability & Nature of Evidence	Key Architectural or Product Decision Supported
				app/layout.tsx and using lifespan context managers in FastAPI 0.137+ / Starlette 1.0+.
SR-11	W3C Web Application Manifest Working Draft & MDN Icon Specs	Oct 2026	Official W3C & MDN web standards (High Confidence).	Specifies 19[span_151](start_span)[span_151](end_span)[span_158](start_span)[span_158](end_span)2x192 and 512x512 PNG icons plus dedicated "purpose": "maskable" icons keeping artwork within the inner 80% (409x409 circle) safe zone.

Document C: Information Architecture and Complete Screen Catalog

Global Navigation Model and Responsive Hierarchy

Swiftradish employs an adaptive three-column spatial architecture on desktop viewports (\ge 1280\text{px}), a collapsible rail and main canvas on tablet viewports (768\text{px} - 1279\text{px}), and a thumb-zone-optimized bottom navigation bar paired with contextual top headers on mobile viewports (< 768\text{px}).

On desktop viewports, the left navigation rail (260\text{px} fixed width) houses the brand header with an interactive Pip mascot status indicator alongside primary navigation links for **Home** (/home), **Explore & Patches** (/explore), **Co-Plots** (/plots), **Messages** (/messages with live unread badge), **Notifications** (/notifications with unread badge), **Bookmarks** (/bookmarks), and **Profile** (/u/[username]), anchored by a primary crimson action button (**"+ Plant a Post / Plot"**) and a bottom **Settings & Safety** (/settings) link. The center canvas (640\text{px} fluid maximum width) renders the primary feed, Patch view, post detail thread, or active chat conversation. The right contextual panel (340\text{px} sticky width) adapts dynamically to the active route: on /home, it displays upcoming Co-Plots and active Radish Patches; on /p/[postId], it renders the Co-Plot RSVP roster and shared carousel contributor queue; and on /messages/[conversationId], it houses the participant directory and shared media grid. On mobile viewports (< 768\text{px}), a persistent bottom tab bar (64\text{px} height plus

env(safe-area-inset-bottom)) exposes five primary destinations—**Home**, **Explore**, **+ Create** (opening a full-screen composer sheet), **Messages**, and **Profile**—while Notifications and Search reside in the top app bar to prevent horizontal tab crowding.

Route Inventory and Deep-Link Resolution

All routes follow deterministic Next.js App Router conventions (src/app/...) partitioned into four logical route groups: (public), (auth), (main), and (legal). Every content artifact (Post, Co-Plot, Patch, User Profile) exposes a canonical shareable URL that renders server-side OpenGraph metadata (og:title, og:description, og:image, og:url, og:type) using absolute image URLs so shared links unfurl accurately across external chat and social platforms. Unauthenticated visitors navigating to a public deep link (/p/[postId], /u/[username], /patches/[slug]) receive a complete server-rendered read-only view accompanied by an unobtrusive bottom banner that prompts authentication only when the visitor clicks an interactive mutation control such as React, Join Plot, or Message.

Comprehensive Screen Catalog and Component Specifications

To achieve complete functional coverage across public, onboarding, feed, creation, coordination, messaging, discovery, moderation, and error surfaces without manufacturing redundant filler pages, the architecture defines **40 canonical screens and modal surfaces** (with explicit desktop/mobile layouts and state variants totaling over 160 distinct UI states).

Screen ID	Route / Entry Point	Primary User Objective	Required Data & Backend Endpoints	Desktop & Mobile Layout Hierarchy	Loading, Empty, Success & Failure States	Auth & Permission Rules	Acceptance Criteria
SCR-01: Public Landing & Manifesto	/ (Unauthenticated)	Understand the Swiftradish thesis and convert to sign-up or explore public Patches.	Featured public Patches & curated sample Co-Plots (GET /api/v1/public/showcase).	**Desktop:** Split editorial layout: left interactive Co-Plot preview card + Pip mascot greeting; right auth entry card. **Mobile:** Stacked vertical showcase with sticky bottom "Join the Patch"	**Loading:** Static SSR shell (zero spinner). **Failure:** Fallback to static curated showcase if API times out.	Public (anon). Redirects to /home if valid session cookie exists.	Renders LCP < 1.2\text{s}; zero fake user counters; interactive demo Co-Plot toggle works locally.

Screen ID	Route / Entry Point	Primary User Objective	Required Data & Backend Endpoints	Desktop & Mobile Layout Hierarchy	Loading, Empty, Success & Failure States	Auth & Permission Rules	Acceptance Criteria
				CTA.			
SCR-02: Public Post / Co-Plot Reader	/p/[postId] (Public view)	View a shared post or Co-Plot via external deep link.	Post payload, media array, public comments (GET /api/v1/posts/{id}).	**Desktop:** Centered 680\text{px} article card with carousel + read-only RSVP count; sticky sign-up bar. **Mobile:** Full-bleed carousel, collapsible comment sheet.	**Loading:** Aspect-ratio-locked media skeleton. **Empty/404:** Pip holding a magnifying glass ("Plot not found or private").	Public if post.visibility == 'public' and author is not private; else 404.	Serves complete OpenGraph meta tags; clicking React/Join opens Auth Modal (SCR-04).
SCR-03: Public Patch Showcase	/patches/[slug] (Public view)	Preview a topic community before registering.	Patch metadata, rules, recent public posts (GET /api/v1/patches/{slug}).	**Desktop:** Banner header, 2-column media grid, right sidebar rules card. **Mobile:** Compact header + single-column feed.	**Empty:** "This Patch is freshly planted—sign in to add the first post."	Public read-only; chat channels hidden until authenticated and joined.	Displays real member count and public posts; hides member directory from unauthenticated scrapers.
SCR-04: Sign-Up & Age Verification	/register (or Auth Modal)	Create an account with 18+ DPDP compliance.	POST /api/v1/auth/register + Supabase Auth OAuth/Email.	Centered 440\text{px} surface over dark botanical backdrop; DOB picker + email/password or Google/Ap	**Success:** Transitions to SCR-07 (Verify) or SCR-08 (Handle Claim). **Failure:** Inline field validation; generic	Public (anon). Rejects DOB indicating age < 18.	Enforces 18+ check client- and server-side; sets HttpOnly, Secure, SameSite=Lax session cookies.

Screen ID	Route / Entry Point	Primary User Objective	Required Data & Backend Endpoints	Desktop & Mobile Layout Hierarchy	Loading, Empty, Success & Failure States	Auth & Permission Rules	Acceptance Criteria
				ple OAuth.	error for existing email to prevent enumeratio n.		
SCR-05: Login Surface	/login	Authenticat e returning user.	Supabase Auth sign-in + GET /api/v1/user s/me.	Centered 440\tex[spa n_304](star t_span)[spa n_304](end _span)t{px} card; password visibility toggle; "Forgot password?" link.	**Loading:** Button morphs to spinner, inputs disabled. **Failure:** Rate-limit banner after 5 failed attempts; Pip "confused" pose.	Public (anon).	Restores intended deep-link via validated internal next query param (blocks open redirects).
SCR-06: Password Recovery & Reset	/auth/reset-password	Request reset token and set new password.	Supabase recovery flow + POST /api/v1/auth /password-r eset.	Two-step card: Step 1 (Email input), Step 2 (New password + strength meter via token link).	**Success:** Toast confirmatio n + auto-redire ct to /login after 2s. **Failure:** Expired token state with 1-tap "Resend link".	Public (anon) with valid single-use recovery token for Step 2.	Always shows identical success message on email request regardless of whether email exists.
SCR-07: Email Verificatio n Gate	/auth/verify	Confirm email ownership via 6-digit OTP or magic link.	POST /api/v1/auth /verify-otp.	6-box auto-advan cing OTP input with 60s resend countdown timer.	**Success:** Pip "sprouting" celebration animation \rightarrow routes to /onboardin	Authenticat ed session with email_verifi ed == false.	Supports paste of 6-digit code; locks out after 5 wrong OTP guesses for 15 minutes.

Screen ID	Route / Entry Point	Primary User Objective	Required Data & Backend Endpoints	Desktop & Mobile Layout Hierarchy	Loading, Empty, Success & Failure States	Auth & Permission Rules	Acceptance Criteria
					g/profile.		
SCR-08: Onboarding Step 1 — Handle & Identity	/onboarding/profile	Claim unique @username, display name, avatar, and theme accent.	GET /api/v1/users/check-username, PATCH /api/v1/users/me.	Split preview: left form controls + avatar cropper; right live preview of the user's Profile Card.	**Loading:** Debounced (300\text{ms}) handle availability badge. **Failure:** Suggests 3 available handle variants if taken.	Authenticated (onboarding_step == 1).	Validates /^[a-z0-9_]{3,24}$/; avatar uploads directly to R2 and strips EXIF before commit.
SCR-09: Onboarding Step 2 — Interest & Patch Picker	/onboarding/interests	Select \ge 3 topics and join initial Radish Patches to prevent cold-start empty feeds.	GET /api/v1/onboarding/topics, POST /api/v1/onboarding/complete.	Interactive bento grid of topic pills and starter Patches with sample visual previews.	**Success:** Unlocks "First Sprout" badge and routes to /home with populated feed.	Authenticated (onboarding_step == 2).	Requires \ge 3 selections (or explicit "Skip to blank feed" secondary button).
SCR-10: PWA Install & Push Primer Drawer	Contextual Bottom Drawer on /home	Guide mobile users (especially iOS Safari) to install PWA and enable Web Push.	Browser beforeinstallprompt (Android/Desktop) or iOS User-Agent detection; POST /api/v1/push/subscribe.	Slide-up drawer showing visual step-by-step for iOS ("Tap Share \rightarrow Add to Home Screen") or 1-tap install on Android.	**Success:** Registers VAPID PushSubscription when launched in standalone display mode.	Authenticated; triggered only after first meaningful interaction (never on cold page load).	Never prompts native Notification.requestPermission() without prior click on custom primer button.
SCR-11: Home Feed — "My Patch" (Chronolo	/home?tab=following	Catch up on posts and active Co-Plots from followed	GET /api/v1/feed/following?cursor=...	**Desktop:** Center 640\text{px} stream + top sticky tab	**Loading:** 3 structural post skeletons matching card	Authenticated.	Strictly chronological (created_at DESC, id DESC);

Screen ID	Route / Entry Point	Primary User Objective	Required Data & Backend Endpoints	Desktop & Mobile Layout Hierarchy	Loading, Empty, Success & Failure States	Auth & Permission Rules	Acceptance Criteria
gical)		accounts and joined Patches.		switcher + right upcoming Co-Plots widget. **Mobile:** Full-width stream + pull-to-refresh.	hierarchy. **Empty:** Honest onboarding card prompting Patch exploration (zero fake posts).		renders "You're all caught up" divider at last_seen_post_id.
SCR-12: Home Feed — "Fresh Harvest" (Discovery)	/home?tab=discover	Discover high-signal posts and Co-Plots outside the immediate follow graph.	GET /api/v1/feed/discover?cursor=...	Identical shell to SCR-11, but each post displays a top recommendation provenance pill ("Because you joined #urban-sketching").	**Empty:** Falls back to trending public Patches if topic filters are too narrow.	Authenticated.	Every card exposes "Why am I seeing this?" popover and 1-tap "Hide topic / Mute author" action.
SCR-13: Post & Co-Plot Detail View	/p/[postId]	View full-resolution carousel, participate in Co-Plot RSVP/contributions, and read/write nested comments.	GET /api/v1/posts/{id}, GET /api/v1/posts/{id}/comments?cursor=...	**Desktop:** Left/Center media stage + Co-Plot coordination block + nested 3-level comment tree; Right contributor/RSVP panel. **Mobile:** Vertical scroll with	**Loading:** Immediate optimistic transition from feed card cache while comments stream in. **Failure:** Inline retry for comment load failure.	Authenticated (or public read-only via SCR-02).	Deep-links to specific comment (#comment-{id}) with highlight pulse; supports keyboard arrow carousel navigation.

Screen ID	Route / Entry Point	Primary User Objective	Required Data & Backend Endpoints	Desktop & Mobile Layout Hierarchy	Loading, Empty, Success & Failure States	Auth & Permission Rules	Acceptance Criteria
				sticky bottom comment input bar.			
SCR-14: Media Lightbox & Carousel Inspector	Modal Overlay (/p/[postId]?media=[index])	Inspect full-resolution images, read full alt-text, and see slide-level contributor credits.	Hydrated from parent post media array (media_assets).	Dark backdrop (rgba(9,10,15,0.95)), pinch-to-zoom / pan canvas, thumbnail filmstrip at bottom, alt-text badge toggle.	**Loading:** Blurhash placeholder transitions to full WebP/AVIF asset.	Matches parent post visibility.	Focus-trapped modal; Escape or swipe-down closes and restores exact scroll position.
SCR-15: Unified Post & Co-Plot Composer	Modal (/compose) or inline top card	Create rich text, multi-image carousels, and interactive Co-Plots.	POST /api/v1/media/upload-intents, POST /api/v1/posts, PUT /api/v1/drafts.	Tabbed composer: **Standard Post** vs **Co-Plot Activity**; drag-and-drop media tray (up to 10 files); per-file progress rings; visibility selector (Public, Followers, Mutuals, Patch).	**Uploading:** Determinate progress bar + cancel button per file. **Success:** Clears composer state, resets file inputs, displays toast with "View Post", ready for next upload. **Failure:** Retains text/completed files, offers 1-tap	Authenticated (active account status).	Submit button remains disabled until all attached media reach READY status; auto-saves drafts to IndexedDB.

Screen ID	Route / Entry Point	Primary User Objective	Required Data & Backend Endpoints	Desktop & Mobile Layout Hierarchy	Loading, Empty, Success & Failure States	Auth & Permission Rules	Acceptance Criteria
					retry on failed file.		
SCR-16: Multi-Image Editor & Alt-Text Drawer	Sub-view inside SCR-15	Crop aspect ratios (1:1, 4:5, 16:9), reorder carousel slides, and write accessible alt-text.	Client-side Canvas/WebWorker manipulation prior to presigned PUT upload.	Grid drag-to-reorder view + slide inspector with aspect-ratio presets and mandatory/assisted alt-text input field.	**Success:** Commits cropped blob to upload queue.	Authenticated.	Strips EXIF GPS/camera serial data client-side and verifies removal server-side.
SCR-17: Co-Plot Contribution Approval Queue	/p/[postId]/manage (Drawer)	Allow Co-Plot host to review and approve/reject media slides submitted by participants.	GET /api/v1/plots/{id}/submissions, POST /api/v1/plots/{id}/submissions/{subId}/decide.	Pending grid of submitted photos with contributor badge and 1-tap "Approve to Carousel" or "Decline" buttons.	**Empty:** Pip relaxing ("No pending slide submissions right now").	Authenticated; restricted strictly to post.author_id or designated co-hosts.	Approved slides immediately append to the parent post's carousel and notify the contributor.
SCR-18: Co-Plots Hub & Calendar	/plots	Track all upcoming, active, and past Co-Plots the user is hosting or attending.	GET /api/v1/plots/my-schedule?status=upcoming|past.	Segmented view: **Upcoming RSVPs**, **Hosting**, and **Shared Memory Archive** (completed plots with collaborative carousels).	**Empty:** "No active Co-Plots on your radar—join one in your Patches or plant your own!"	Authenticated.	Exports .ics calendar file for scheduled plots; links directly to each plot's dedicated group chat.
SCR-19: Explore & Discovery	/explore	Browse curated Radish	GET /api/v1/explore/hub.	Search command bar at top;	**Loading:** Bento grid skeletons.	Authenticated.	Excludes blocked/muted users

Screen ID	Route / Entry Point	Primary User Objective	Required Data & Backend Endpoints	Desktop & Mobile Layout Hierarchy	Loading, Empty, Success & Failure States	Auth & Permission Rules	Acceptance Criteria
Directory		Patches, active public Co-Plots, and rising community creators.		horizontal carousels for "Featured Patches", "Happening This Weekend (Co-Plots)", and topic category grid.			and quarantined content by default.
SCR-20: Unified Search Results & Command Palette	/search?q=[query]&type=[all|users|posts|patches]	Execute fast full-text and fuzzy trigram searches across the platform.	GET /api/v1/search?q=...&type=...&cursor=...	Instant typeahead popover (Cmd/Ctrl+K) + full tabbed results page (All, People, Co-Plots & Posts, Patches, Tags).	**Loading:** Subtle top shimmer bar. **Empty:** Pip shrugging with spelling suggestions powered by pg_trgm similarity. **Failure:** Error banner with retry button.	Authenticated.	Debounces input by 250\text{ms}; aborts stale in-flight AbortController requests on rapid typing.
SCR-21: Radish Patch (Community Space) Home	/patches/[slug]	Participate in a topic community's curated feed, active Co-Plots, and real-time chat channels.	GET /api/v1/patches/{slug}, GET /api/v1/patches/{slug}/posts.	Header banner with Patch rules & Join button; sub-navigation tabs: **Curated Feed**, **Patch Co-Plots**, and **Live Lounge**	**Empty:** Prompt for founding members to plant the first topic post.	Authenticated; public Patches readable by all, posting requires membership.	Joining a Patch updates feed subscriptions immediately and unlocks the Patch's chat channels.

Screen ID	Route / Entry Point	Primary User Objective	Required Data & Backend Endpoints	Desktop & Mobile Layout Hierarchy	Loading, Empty, Success & Failure States	Auth & Permission Rules	Acceptance Criteria
				(up to 3 chat channels).			
SCR-22: Patch Creation & Steward Console	/patches/new & /patches/[slug]/settings	Create a new topic Patch or manage rules, pinned posts, and community reports.	POST /api/v1/patches, PATCH /api/v1/patches/{slug}, GET /api/v1/patches/{slug}/reports.	Form for slug, title, description, cover banner, up to 5 community rules, and moderator roster management.	**Failure:** Slug collision warning with instant alternative suggestions.	Authenticated; creation requires account age \ge 24\text{h} and verified email to prevent spam.	Stewards can remove posts from their Patch (without deleting the post from the author's personal profile).
SCR-23: User Profile & Identity Showcase	/u/[username]	Express personal identity, showcase posts/Co-Plots, display badges, and initiate follow/chat.	GET /api/v1/users/{username}/profile, GET /api/v1/users/{username}/posts.	Custom theme-tinted header, avatar with Pip status frame, bio, verified domain link, earned Badges strip, Follow/Mutual pill, and 3 content tabs: **Posts**, **Co-Plots**, **Remixes**.	**Private Account State:** Shows header + "This Patch is Private. Send a follow request to collaborate." **Blocked State:** Minimal shell hiding posts and actions.	Public or Authenticated; enforces private-account and block-list visibility checks server-side.	Renders custom dark-safe accent token chosen by user; clicking badge opens milestone provenance popover.
SCR-24: Profile Editor & Theme Customizer	/settings/profile	Edit bio, avatar, header, privacy mode, and select profile theme tokens.	PATCH /api/v1/users/me/profile.	Form with live real-time preview card; curated dark-first accent swatches	**Success:** Toast confirmation + instant cache invalidation on /u/[username].	Authenticated (owner only).	All theme presets guarantee \ge 4.5:1 text contrast automatically.

Screen ID	Route / Entry Point	Primary User Objective	Required Data & Backend Endpoints	Desktop & Mobile Layout Hierarchy	Loading, Empty, Success & Failure States	Auth & Permission Rules	Acceptance Criteria
				(Radish Crimson, Electric Lime, Cyber Teal, Amber Glow, Violet Dusk).			
SCR-25: Relationships Manager (Followers, Following, Mutuals & Requests)	/u/[username]/relations?tab=[mutuals|followers|following|requests]	Manage social graph connections and approve/decline pending follow requests.	GET /api/v1/users/{id}/relations, POST /api/v1/relations/requests/{id}/decide.	Tabbed user list with instant filter search, mutual indicator badge, and Approve/Decline or Follow/Unfollow buttons.	**Empty:** Contextual message per tab.	requests tab strictly visible only to profile owner; other tabs respect account privacy settings.	Unfollowing a private account prompts a confirmation dialog warning that re-following will require approval.
SCR-26: Bookmarks & Saved Collections	/bookmarks	Organize saved posts and Co-Plots into private personal folders.	GET /api/v1/bookmarks?collection_id=...	Left/top collection filter pills (All Saved, Weekend Ideas, Design Inspo, + New Folder) + masonry/list post grid.	**Empty:** "Bookmark posts or Co-Plots to save ideas for future doing."	Authenticated (strictly private to owner).	Bookmarking is private—authors do not receive notifications showing *who* bookmarked their post.
SCR-27: Messages Inbox & Split Workspace	/messages	View active 1:1 chats, group chats, and Co-Plot coordination threads.	GET /api/v1/conversations?cursor=... + Root WebSocket subscription.	**Desktop:** Dual-pane workspace (340\text{px} conversation list on left + active	**Empty:** Pip waving ("Start a chat with a mutual or join a Co-Plot thread").	Authenticated.	Sorts conversations deterministically by last_message_at DESC;

Screen ID	Route / Entry Point	Primary User Objective	Required Data & Backend Endpoints	Desktop & Mobile Layout Hierarchy	Loading, Empty, Success & Failure States	Auth & Permission Rules	Acceptance Criteria
				chat canvas on right). **Mobile:** Full-screen conversation list with pinned threads at top.			updates unread badges in real time via WebSocket.
SCR-28: Active Conversation Canvas (1:1, Group & Co-Plot Chat)	/messages/[conversationId]	Exchange real-time messages, media attachments, replies, and reactions.	GET /api/v1/conversations/{id}/messages?cursor=... + WebSocket frames (msg.send, msg.ack, typing.pulse).	Sticky top bar (participant info, Co-Plot context banner, connection health pill); virtualized reverse-scroll message stream; rich bottom composer with attachment tray.	**Sending:** Optimistic message bubble with clock icon + client_msg_id. **Delivered/Read:** Checkmark indicator. **Disconnected:** Amber banner "Reconnecting..." while queuing outgoing messages locally.	Authenticated; requires active row in conversation_participants.	Supports inline reply quoting, 15-minute edit window, soft delete, and image uploads with zero duplicate rendering on reconnect.
SCR-29: Message Requests & Spam Quarantine	/messages/requests	Safely triage messages from non-mutuals without exposing read receipts or loading untrusted media.	GET /api/v1/conversations/requests, POST /api/v1/conversations/{id}/accept|decline.	Quarantined list; opening a request blurs images by default and disables read receipts/typing indicators	**Empty:** "No pending message requests."	Authenticated.	"Decline & Block" deletes the request and blocks the sender in a single atomic transaction without notifying the sender.

Screen ID	Route / Entry Point	Primary User Objective	Required Data & Backend Endpoints	Desktop & Mobile Layout Hierarchy	Loading, Empty, Success & Failure States	Auth & Permission Rules	Acceptance Criteria
				until "Accept" is clicked.			
SCR-30: Group & Co-Plot Chat Inspector Drawer	Right Drawer inside SCR-28	Manage participants, mute notifications, view shared media, or leave/report the group.	GET /api/v1/conversations/{id}/details, PATCH /api/v1/conversations/{id}/m[span_15](start_span)[span_15](end_span)[span_18](start_span)[span_18](end_span)embership.	Member roster with role badges (Host, Member), notification mute duration selector (1h, 8h, Forever), and grid of shared chat images.	**Success:** Instant state reflection on mute/pin toggles.	Authenticated participant; kick/invite actions restricted to group host/admins.	Leaving a group inserts a system event message and revokes future WebSocket room subscription immediately.
SCR-31: Activity Center (Notifications Hub)	/notifications	Review grouped social interactions, Co-Plot updates, and system alerts.	GET /api/v1/notifications?filter=[all|plots|mentions], POST /api/v1/notifications/mark-read.	Filter pills (All, Co-Plots & RSVPs, Mentions & Replies, System); grouped notification rows with unread crimson dot indicator and media thumbnail preview.	**Empty:** Pip sleeping peacefully ("All quiet in the patch").	Authenticated.	Clicking any notification marks it read and deep-links to the exact originating post, comment, or Co-Plot.
SCR-32: Badges, Milestones & Pip Showcase	/achievements (or modal from Profile)	View earned contribution milestones, badge criteria, and	GET /api/v1/users/me/badges.	Visual showcase grid showing earned badges	**Success:** Replayable Lottie celebration animation on newly	Authenticated.	Allows user to pin up to 3 earned badges to their public profile

Screen ID	Route / Entry Point	Primary User Objective	Required Data & Backend Endpoints	Desktop & Mobile Layout Hierarchy	Loading, Empty, Success & Failure States	Auth & Permission Rules	Acceptance Criteria
		unlocked Pip mascot poses.		with unlock dates and transparent criteria for locked milestones (no hidden dark-pattern bars).	unlocked badges.		header.
SCR-33: Settings — Account, Sessions & Security	/settings/account	Manage email, password, active devices, and initiate account export or deletion.	GET /api/v1/auth/sessions, DELETE /api/v1/auth/sessions/{id}, POST /api/v1/account/delete-request.	Clean settings list: Change Email/Password, Active Sessions list (browser, IP city, last active + "Revoke" button), Data Export, and Danger Zone (Delete Account).	**Confirmation:** Account deletion requires re-entering password/OTP and typing "DELETE" in modal (SCR-37).	Authenticated with fresh session requirement for destructive mutations.	Revoking a session immediately invalidates its refresh token and drops any active WebSocket for that session.
SCR-34: Settings — Privacy, Safety & Blocked/Muted Lists	/settings/privacy	Control account visibility, DM permissions, search indexability, and manage blocked/muted accounts.	PATCH /api/v1/users/me/privacy, GET /api/v1/safety/blocks, DELETE /api/v1/safety/blocks/{userId}.	Toggles for Private Account, Who Can Message Me (Mutuals Only, Anyone via Requests, No One), Hide from Search Engines,	**Success:** Optimistic toggle with toast confirmation.	Authenticated.	Switching from Private to Public prompts confirmation noting that all pending follow requests will be auto-approved.

Screen ID	Route / Entry Point	Primary User Objective	Required Data & Backend Endpoints	Desktop & Mobile Layout Hierarchy	Loading, Empty, Success & Failure States	Auth & Permission Rules	Acceptance Criteria
				and tabbed Blocked/Muted lists with "Unblock" actions.			
SCR-35: Settings — Notification & PWA Preferences	/settings/notifications	Configure granular push, email, and in-app notification rules plus quiet hours.	GET /api/v1/notifications/preferences, PUT /api/v1/notifications/preferences.	Matrix of toggles (In-App, Web Push, Email) per event category + Quiet Hours time range picker + Reduced Motion override toggle.	**Permission Warning:** Shows browser instruction banner if OS/browser push permission is currently denied.	Authenticated.	Respects quiet hours server-side in the notification worker before dispatching Web Push payloads.
SCR-36: Universal Report & Statutory Grievance Modal	Modal Overlay (ReportModal)	Report abusive content/users or file a formal statutory grievance under India IT Rules 2021.	POST /api/v1/safety/reports.	Multi-step wizard: Step 1 (Category selection: Harassment, Spam, Hate Speech, Privacy Violation / Impersonation [24h SLA], Copyright/Legal); Step 2 (Context notes + optional immediate Block/Mute	**Success:** Displays copyable Ticket ID, SLA timeline (24h acknowledgment, 24h/72h/15d resolution), and prompts optional one-click Block.	Authenticated (also accessible via public statutory form at /legal/grievance for non-users).	Immediately hides the reported post/comment from the reporter's client view upon submission.

Screen ID	Route / Entry Point	Primary User Objective	Required Data & Backend Endpoints	Desktop & Mobile Layout Hierarchy	Loading, Empty, Success & Failure States	Auth & Permission Rules	Acceptance Criteria
				checkboxes); Step 3 (Confirmation with unique Tracking Ticket ID).			
SCR-37: Account Deletion & Data Purge Flow	Modal / /settings/delete-account	Execute transparent, GDPR/DPDP-compliant account deletion with 14-day grace window or immediate media purge.	POST /api/v1/account/schedule-deletion.	Explains exact consequences: profile hidden immediately, sessions revoked, media objects and database rows permanently purged via background worker, chat messages anonymized.	**Success:** Logs user out across all devices and routes to a goodbye confirmation screen with grace-period recovery instructions.	Authenticated + step-up credential check.	Enqueues durable cleanup job (account.purge_media) to delete R2 objects after grace window expires.
SCR-38: User Safety & Grievance Status Tracker	/settings/reports	Track the status of submitted reports, statutory grievances, or appeal a moderation action against one's own content.	GET /api/v1/safety/my-reports, POST /api/v1/safety/appeals.	Timeline list of submitted tickets showing status (Received - Ack Sent, Under Review, Action Taken, Closed) + Appeal	**Empty:** "You have no active safety reports or moderation notices."	Authenticated.	Satisfies Rule 3(1) and Rule 4(8) of IT Rules 2021 requiring ticket tracking and user opportunity to dispute content disablement.

Screen ID	Route / Entry Point	Primary User Objective	Required Data & Backend Endpoints	Desktop & Mobile Layout Hierarchy	Loading, Empty, Success & Failure States	Auth & Permission Rules	Acceptance Criteria
				submission form for content removals.			
SCR-39: Trust & Safety Moderation Triage Console (Admin/Steward)	/internal/moderation	Allow authorized moderators and the Resident Grievance Officer to review reports within statutory SLAs.	GET /api/v1/admin/moderation/queue?priority=sla_24h, POST /api/v1/admin/moderation/cases/{id}/resolve.	Priority-sorted queue highlighting 24h-SLA items in crimson at the top; side-by-side evidence snapshot + 1-key action macros (Remove Content, Suspend User, Purge Media, Dismiss).	**Empty:** "SLA queue clear."	Authenticated with role IN ('admin', 'grievance_officer', 'moderator') + MFA enforced.	Every moderation decision writes an immutable row to audit_events and triggers user notification with dispute link.
SCR-40: Global Error, Offline, 404 & Rate-Limit Surfaces	not-found.tsx, error.tsx, /offline	Provide honest recovery paths during network loss, missing routes, or 429 rate limiting.	Service Worker cache for /offline.	Expressive Pip mascot illustration (e.g., Pip tangled in roots for 500 error; Pip holding a watering can for 429 cooldown with live countdown timer) + "Retry" or "Return Home" CTA.	Self-contained error/offline states.	Public / Authenticated.	/offline page is pre-cached by the Service Worker at install time and allows reading cached /bookmarks and recent feed items.

Document D: Interaction Flows and Deterministic State Specifications

End-to-End User Journey Specifications

To eliminate interface ambiguity—such as file uploads remaining stuck in an indeterminate state or reconnecting chat sockets rendering duplicate messages—all primary user journeys and subsystem state machines are governed by deterministic state transitions. Every user action must transition the interface to an explicit, inspectable state within defined latency and animation budgets.

Registration, Age Gate, and Cold-Start Onboarding Flow

When an unauthenticated user opens /register (SCR-04), the view initializes in the IDLE state, presenting inputs for email, password, and Date of Birth (DOB). Upon submission, the flow enters VALIDATING. If the calculated age is under 18, the state transitions immediately to AGE_REJECTED, blocking account creation in compliance with Section 9 of the DPDP Act 2023 and rendering an informative policy explanation without storing personal data. If the user is 18 or older, the client dispatches POST /api/v1/auth/register and transitions to AWAITING_OTP (SCR-07), displaying a six-digit one-time passcode input with a 60-second resend timer. Once the user submits a valid OTP (POST /api/v1/auth/verify-otp), the session upgrades to authenticated, Pip plays a 450\text{ms} sprouting animation, and the router advances to PROFILE_SETUP (SCR-08). Here, a 300\text{ms} debounced check (GET /api/v1/users/check-username) validates handle availability in real time, offering three available variants if a collision occurs. After saving the profile (PATCH /api/v1/users/me), the flow enters INTEREST_SEEDING (SCR-09), where selecting at least three topics or starter Radish Patches materializes initial feed subscriptions, grants the **"First Sprout"** badge, and routes the user to a populated /home feed (SCR-11).

Multi-Image and Video Upload, Composer Lifecycle, and Recovery Flow

A common defect in rich web applications occurs when a user uploads media, publishes a post, and attempts to start a second post—only to encounter stale file references, disabled inputs, or orphaned progress bars. Swiftradish enforces a strict eight-state composer lifecycle across single-image, multi-image carousel, and video uploads:

Composer / Media State	Triggering Event	Exact Visual UI Representation	Allowed User Actions	Backend / Storage State & Cleanup Rules
1. COMPOSER_EMPTY	User opens composer (SCR-15) or completes/resets a prior post.	Text area focused; media tray hidden; <input type="file"> value explicitly reset to ""; Publish button disabled (opacity-40).	Type text, select files (up to 10 images), toggle Co-Plot mode.	No active upload intents. Local IndexedDB draft synced if text is typed.
2.	User selects N	Instant local	Remove individual	Client checks

Composer / Media State	Triggering Event	Exact Visual UI Representation	Allowed User Actions	Backend / Storage State & Cleanup Rules
CLIENT_VALIDATING	files via file picker or drag-and-drop.	thumbnail preview generated via URL.createObjectURL(); Pip inspecting badge shown for < 150\text{ms}.	preview tile, crop/edit (SCR-16), add alt-text.	MIME magic bytes (FF D8 FF JPEG, 89 50 4E 47 PNG, 52 49 46 46 WebP), size (\le 15\text{MB} image, \le 100\text{MB} video), and strips EXIF GPS tags via Canvas/WebWorker. Invalid files trigger immediate toast error and are excluded from queue.
3. INTENT_AUTHORIZED	Validated files request upload slots via POST /api/v1/media/upload-intents.	Thumbnail displays circular progress ring at 5%. Publish button shows label "Uploading media..." (disabled).	Cancel individual file upload, edit caption/alt-text while upload runs in parallel.	Backend creates media_assets rows with status = 'PENDING_UPLOAD', sets expires_at = NOW() + 1 hour, and returns short-lived (15\text{min}) S3-compatible presigned PUT URLs for Cloudflare R2.
4. UPLOADING_PROGRESS	Browser XMLHttpRequest / fetch streams bytes directly to R2 presigned URL.	Smooth determinate progress ring (5% \rightarrow 90%) updated via xhr.upload.onprogress; X button overlays corner of tile.	Click X to abort (xhr.abort()), which removes tile and calls DELETE /api/v1/media/{id}.	Bytes stream directly from browser to Cloudflare R2 edge without blocking FastAPI workers.
5. VERIFYING_PROCESSING	R2 returns HTTP 200 OK on PUT; client calls POST /api/v1/media/{id}/complete.	Progress ring hits 95% and pulses softly with label "Verifying...".	Edit text, reorder tiles, edit alt-text.	FastAPI verifies object existence and exact byte size via R2 HeadObject, reads first 4\text{KB} to

Composer / Media State	Triggering Event	Exact Visual UI Representation	Allowed User Actions	Backend / Storage State & Cleanup Rules
				verify file signature, extracts dimensions + Blurhash, and sets status = 'READY' (or enqueues arq job for responsive variants/video transcode).
6. UPLOAD_FAILED	Network drop, R2 timeout, or server validation mismatch on 1 or more files.	Failed tile displays crimson border (#FF4D5E), error badge, and two inline buttons: **"Retry"** and **"Remove"**. Other succeeded tiles remain intact at READY.	Click **Retry** (re-initiates from State 3/4 for *only* the failed file) or **Remove** (clears failed tile so post can be published with remaining valid files).	Failed media_assets row marked FAILED; background arq cron purges unattached PENDING_UPLOAD / FAILED objects after 6 hours.
7. MEDIA_READY	All attached tiles have reached status = 'READY'.	Green checkmark badge (#10B981) fades into corner of each thumbnail (200\text{ms}); Publish button becomes active (#FF4D5E).	Click **"+ Add more"** (up to 10 total), reorder slides, or click **"Plant Post"**.	All media_assets rows exist in PostgreSQL with status = 'READY' and post_id = NULL owned by current_user.id.
8. POST_COMMITTED	User clicks **"Plant Post"** (POST /api/v1/posts) and receives HTTP 201 Created.	Composer executes a 250\text{ms} scale-down transition; Pip plays a joyful "harvest" micro-animation in a bottom toast ("Post planted! View"). **Crucially:** All composer state arrays (attachments, text, coPlotConfig) are wiped, all URL.revokeObject	Click "View" in toast to navigate to /p/[postId], or immediately begin typing/uploading a new post without page reload.	PostgreSQL transaction atomically inserts posts row and updates media_assets SET post_id = new_post.id WHERE id IN (...) AND owner_id = current_user.id AND status = 'READY'. Local IndexedDB draft is deleted.

Composer / Media State	Triggering Event	Exact Visual UI Representation	Allowed User Actions	Backend / Storage State & Cleanup Rules
		URL() cleanup calls fire, <input type="file"> value is reset to "", and the composer returns cleanly to **COMPOSER_EMPTY**, immediately ready for a subsequent upload.		

If the final POST /api/v1/posts request fails due to a transient network drop after media files have already reached MEDIA_READY, the composer transitions to POST_SUBMIT_ERROR rather than discarding the uploaded work: all READY media asset IDs remain bound in the composer state and persisted in the local IndexedDB draft so clicking **"Retry Publish"** commits the post instantaneously without re-uploading a single byte.

Real-Time Messaging, Optimistic UI, and Network Interruption Recovery Flow

When a user composes a chat message in SCR-28 and presses Enter, the client transitions the message into OPTIMISTIC_QUEUED: it generates a UUIDv4 client_msg_id, appends the message bubble immediately to the local Zustand and TanStack Query state with a subtle clock icon (delivery_state = 'sending'), and transmits a WebSocket frame ({"type": "msg.send", "conversation_id": "...", "client_msg_id": "...", "body": "..."}). Upon receiving the frame, the FastAPI WebSocket handler verifies that the authenticated user holds an active row in conversation_participants and is not blocked by the recipient, then executes an idempotent SQL insert (INSERT INTO messages ... ON CONFLICT (conversation_id, sender_id, client_msg_id) DO NOTHING RETURNING *). FastAPI immediately returns a msg.ack frame mapping client_msg_id to the canonical time-ordered UUIDv7 server_msg_id and created_at timestamp—transitioning the sender's UI bubble to PERSISTED_DELIVERED (single checkmark)—while publishing msg.new to the Redis Pub/Sub channel for all connected participants.

If the client's network connection drops mid-session, the root WebSocket state machine transitions to DISCONNECTED_BUFFERING and displays an amber top banner ("Reconnecting..."). Messages typed while disconnected are placed in a local pendingOutbox queue marked "Waiting for connection". The client attempts reconnection using exponential backoff (1\text{s}, 2\text{s}, 4\text{s}, 8\text{s} plus random jitter) by requesting a fresh 30-second single-use ticket (POST /api/v1/ws/ticket). Immediately upon socket restoration (RECONNECT_SYNC), the client sends {"type": "sync.resume", "conversation_id": "...", "last_received_msg_id": "..."} to fetch any messages created during the outage in strict UUIDv7 order, merges them into the timeline, and then flushes the pendingOutbox queue. Because every queued message retains its original client_msg_id, any message that actually reached the server right before the socket dropped is deduplicated idempotently without creating duplicate

chat bubbles.

Reporting, Blocking, and Statutory Moderation Flow

When a user activates the report workflow (SCR-36) on any post, comment, or message, selecting a violation category and submitting POST /api/v1/safety/reports immediately transitions the local item to REPORTED_HIDDEN, replacing the offending content in the user's viewport with a muted shield ("You [span_84](start_span)[span_84](end_span)reported this content"). On the backend, the service inspects the statutory severity tier: reports involving bodily privacy violations, non-consensual imagery, or identity morphing/impersonation are assigned sla_tier = 'CRITICAL_24H', trigger an immediate webhook alert to the moderation triage console (SCR-39), and dispatch an automated acknowledgment notification and email containing a unique tracking ticket number within 60 seconds (well inside the 24-hour legal mandate under India's IT Rules 2021). If the reporting user simultaneously selects **"Block @username"**, the database executes an atomic transaction that records the block in user_blocks, deletes any existing follow edges in both directions, revokes pending Co-Plot invitations, and filters both users out of each other's feeds, search results, and direct message routes.

Post-Action UI Guarantees and Motion Timing Rules

Motion across Swiftradish is strictly functional—reinforcing spatial origin, confirming state changes, and masking sub-100ms network transitions without ever delaying user input.
| Interaction Category | Animation Purpose | Duration & Easing Curve | Reduced-Motion Behavior (prefers-reduced-motion: reduce) | Interruption & Error Feedback Rules | | :--- | :--- | :--- | :--- | :--- | | **Micro-Interactions (Reactions, Bookmark, Follow Toggle)** | Confirm immediate tactile registration of optimistic state change. | 160\text{ms} — cubic-bezier(0.22, 1, 0.36, 1) (subtle scale 1.0 \rightarrow 1.18 \rightarrow 1.0). | Instant color/fill swap (0\text{ms} scale transform). | Rapid repeated clicks cancel in-flight keyframes and debounce network mutation; on API failure, reverts color with a 180\text{ms} horizontal shake (translateX(-4px, 4px)). | | **Drawer, Sheet & Modal Transitions (Composer, Lightbox)** | Establish spatial hierarchy between parent feed and overlay surface. | 220\text{ms} enter / 180\text{ms} exit — cubic-bezier(0.16, 1, 0.3, 1). | 120\text{ms} opacity cross-fade only; zero vertical slide or scale. | Pressing Escape or swiping down mid-entrance immediately reverses velocity toward closed state without waiting for entrance completion. | | **Toast & Activity Alerts** | Surface non-blocking confirmation (e.g., upload complete, ticket created). | 200\text{ms} enter, 4000\text{ms} dwell, 150\text{ms} exit. | Instant opacity fade in/out. | Hovering or focusing a toast pauses its auto-dismiss timer indefinitely until focus leaves. | | **Pip Mascot Celebrations (Badge Unlock, First Co-Plot)** | Provide emotional warmth and milestone recognition. | 550\text{ms} - 800\text{ms} single-play Lottie/SVG sequence. | Renders static high-resolution keyframe SVG pose; disables looping or bouncing motion. | Clicking anywhere on or outside the badge card immediately skips to the final settled state. |

Document E: Visual Design System, Mascot Taxonomy, and Stitch Execution Plan

Visual Design Directions and Recommended Aesthetic System

Three distinct visual directions were evaluated against Swiftradish's product thesis and target demographics:

Design Direction	Visual Metaphor & Surface Strategy	Typography & Color Profile	Strengths	Weaknesses	Verdict
Direction 1: "Nocturnal Botanical Studio"	Deep, warm obsidian-soil surfaces (#0B0D11 to #181C24) paired with crisp botanical crimson (#FF4D5E) and fresh sprout lime (#2EE59D) accents; subtle 1\text{px} luminance borders (rgba(255,255,255,0.08)).	Display: *Plus Jakarta Sans* (geometric warmth); UI/Body: *Inter*; Metadata/Time stamps: *JetBrains Mono*.	High contrast for photography and video; warm dark mode prevents eye strain without looking like a sterile crypto/SaaS dashboard; pairs naturally with the radish mascot.	Requires disciplined token usage so crimson and lime accents do not clash on the same card.	**RECOMMENDED**
Direction 2: "Sunlit Farmers' Market" (Light-First Editorial)	Warm parchment/cream backgrounds (#FAF7F2) with ink-black serif headings and terracotta cards.	Display: *Fraunces* (serif); Body: *DM Sans*.	Warm, literary feel reminiscent of editorial newsletters or recipe journals.	Poor fit for evening real-time chat, video playback, and media lightbox immersion; feels slow and magazine-like rather than co-active.	Rejected
Direction 3: "Neon Hyper-Glass"	Pure black (#000000) OLED background with heavy backdrop-filter: blur(24px) translucent cards and multi-stop	Display: *Space Grotesk*; Body: *Inter*.	Flashy in static Dribbble mockups.	Severe GPU compositing lag during long feed scrolling on mid-range mobile devices; poor WCAG contrast over user photos; resembles a	Rejected

Design Direction	Visual Metaphor & Surface Strategy	Typography & Color Profile	Strengths	Weaknesses	Verdict
	cyan/magenta glowing borders.			generic AI SaaS template.	

Selecting Direction 1 (**"Nocturnal Botanical Studio"**) grounds the application in a warm, tactile dark mode (#0B0D11 canvas, #12151C primary card surface, #1A1E28 elevated modal and composer surface) that makes user photography and Co-Plot carousels stand out with gallery-grade vibrancy while feeling inviting during evening group coordination. Unlike pure #000000 black backgrounds—which induce halation and text smearing for users with astigmatism—warm carbon-slate surfaces establish depth through progressive surface luminance and crisp 1\text{px} hairline borders rather than heavy drop shadows or GPU-intensive glassmorphism. A companion high-craft Light Theme (**"Daylight Parchment"**, #F6F7F9 canvas with #FFFFFF card surfaces) is also provided via CSS custom properties for high-glare outdoor usage.

Design Tokens, Surface Hierarchy, and Accessibility Standards

All visual tokens are codified into a portable DESIGN.md specification and mapped 1:1 to CSS custom properties in globals.css.

Token Category	CSS Variable Name	Dark Theme ("Nocturnal Studio")	Light Theme ("Daylight Parchment")	WCAG 2.2 AA Contrast & Semantic Usage Rule
Canvas Background	--bg-canvas	#0B0D11	#F6F7F9	Root viewport background; never used for interactive cards.
Surface 1 (Card / Feed)	--bg-surface-1	#12151C	#FFFFFF	Primary post cards, sidebar widgets, chat bubbles (incoming).
Surface 2 (Elevated)	--bg-surface-2	#1A1E28	#EEF0F4	Modals, drawers, composer tray, hover states, active chat input.
Surface 3 (Interactive)	--bg-surface-3	#232936	#E2E6ED	Pill badges, secondary buttons, code blocks, skeleton base.
Border Hairline	--border-subtle	rgba(255, 255, 255, 0.08)	rgba(15, 23, 42, 0.09)	1\text{px} solid structural dividers; zero glowing neon gradients.

Token Category	CSS Variable Name	Dark Theme ("Nocturnal Studio")	Light Theme ("Daylight Parchment")	WCAG 2.2 AA Contrast & Semantic Usage Rule
Text Primary	--text-primary	#F3F5F8	#0F141C	Contrast ratio 16.4:1 (Dark) / 17.1:1 (Light) against Surface 1.
Text Secondary	--text-secondary	#9BA3B4	#4B5565	Contrast ratio 6.8:1 (Dark) / 6.4:1 (Light); used for bios, timestamps, captions.
Primary Brand (Radish)	--accent-primary	#FF4D5E	#E02E40	Primary CTAs, active Co-Plot indicators, unread badges. White text on #E02E40 achieves 4.6:1; on #FF4D5E dark ink #0B0D11 achieves 7.9:1.
Secondary Brand (Leaf)	--accent-secondary	#2EE59D	#0D9F6E	Co-Plot "Joined" status, mutual connection pills, verified domain badges.
Semantic Status	--status-success / --warn / --error / --info	#10B981 / #F59E0B / #EF4444 / #3B82F6	#059669 / #D97706 / #DC2626 / #2563EB	Always paired with an explicit icon and text label (never color alone).

Typography follows a fluid clamp() scale anchored at 16px (1rem) body text with a 1.55 line-height (max-width: 65ch). Headings use *Plus Jakarta Sans* (600/700 weight, -0.02em tracking), while counts, SLA timers, and Co-Plot dates use *JetBrains Mono* with font-variant-numeric: tabular-nums so numeric increments never cause horizontal layout jitter. Post cards are separated by a clean 1\text{px} --border-subtle divider on mobile and a 16\text{px} vertical gap on desktop with a restrained 14px border radius. Co-Plot cards feature a distinct 3\text{px} left --accent-secondary indicator bar and a structured coordination tray so users immediately distinguish an actionable Co-Plot from a standard media post. Every interactive target enforces a minimum hit area of 44 \times 44\text{px} on touch devices and 36 \times 36\text{px} on desktop, paired with a high-contrast :focus-visible ring (2px solid var(--accent-secondary) with 2px offset).

The Swiftradish Mascot System and 216-Pose Structured Taxonomy

Digital mascots succeed when they operate as expressive, systematic micro-illustrations rather

than intrusive clip-art assistants, a pattern validated by Duolingo's token-and-illustration design system. Swiftradish's mascot is **Pip**—a compact, geometric root-vegetable character with a rounded crimson-magenta body (#FF4D5E shading to #C81E32), cream root-tip feet (#FFF5EB), expressive dark eyes with crisp specular highlights, and a three-leaf sprout crown (#2EE59D and #0D9F6E) that acts as a secondary emotional indicator (perking up when excited, drooping when offline, and spinning gently during media processing).

Evaluating asset formats across static raster PNG/WebP, layered vector SVG, Lottie/DotLottie animations, and sprite sheets demonstrates that a **Hybrid Layered-SVG plus Selective DotLottie Architecture** is optimal. Static rasters scale poorly across pixel densities and cannot adapt their keylines to theme changes, while rendering dozens of simultaneous Lottie players in a scrolling feed degrades mobile frame rates. Consequently, all 216 canonical poses are authored as clean, layered **SVG masters** (3\text{KB}-8\text{KB} gzipped, using CSS variables for dark/light outline adaptation) exported alongside 256x256 and 512x512 WebP fallbacks for OpenGraph and push notification payloads. Only the 12 hero celebratory and loading states are additionally compiled into .lottie animations, which play once and automatically fall back to static SVG frames when prefers-reduced-motion: reduce is enabled.

Rather than commissioning 200 arbitrary illustrations, the 216-pose library is structured as a **4-Dimensional Parametric Matrix**:

\text{6 Core Archetypes} \times \text{6 Emotional States} \times \text{3 Action Contexts} \times \text{2 Framing/Seasonal Variants} = 216 \text{ Canonical Poses}

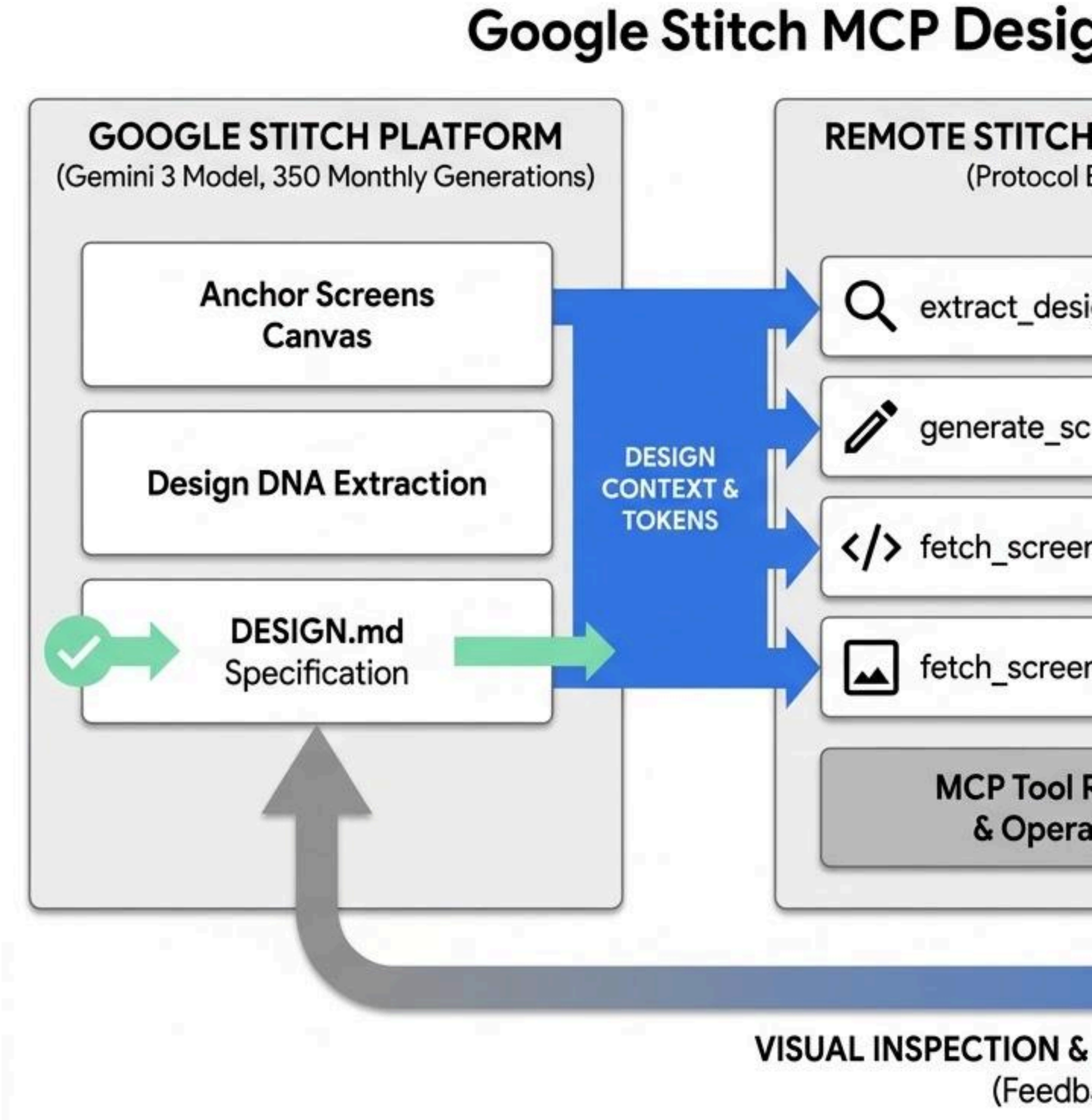
Dimension 1: Core Archetype (6)	Dimension 2: Emotional State (6)	Dimension 3: Product Action Context (3)	Dimension 4: Framing / Variant (2)	Naming Convention & Metadata Schema
01_sprout (Onboarding & New Beginnings)	joyful (Wide smile, perky leaves)	idle (Ambient state / empty view)	v1_standard (Full body 3/4 angle, universal)	Format: pip_{archetype}_{emotion}_{context}_{variant}.svg
02_coplot (Social Coordination & Doing)	curious (Head tilt, one leaf raised)	active (In-progress / uploading / syncing)	v2_closeup (Bust/sticker framing with seasonal prop or badge)	Example: pip_coplot_hype_complete_v1.svg
03_creative (Camera, Carousel & Remixing)	hype (Starry eyes, jumping silhouette)	complete (Success / milestone unlocked)	—	Viewbox: 0 0 256 256 with 16\text{px} outer padding safe zone.
04_chat (Messaging, Wave & Presence)	chill (Peaceful closed eyes, cozy posture)	—	—	Dark/Light outline adapter: stroke: var(--pip-keyline, #181C24)
05_system (Loading, Offline, Rate-Limit, 404)	oops (Sweat drop, tangled roots, apologetic)	—	—	JSON manifest stores pose_id, alt_text, reduced_motion_fallback_svg, and lottie_url.
06_safety (Privacy	focused	—	—	100% original

Dimension 1: Core Archetype (6)	Dimension 2: Emotional State (6)	Dimension 3: Product Action Context (3)	Dimension 4: Framing / Variant (2)	Naming Convention & Metadata Schema
Shield, Block, Moderation)	(Determined brow, protective stance)			internal vector IP licensed under project repository ownership.

To prevent illustration production from delaying core application engineering, the MVP launches with a **30-Pose Core Set** covering all essential UI states: 4 Onboarding poses, 6 Empty-State poses (Feed, Search, Bookmarks, Chat, Plots, Notifications), 4 Upload/Composer states (Inspecting, Uploading, Success, Error), 4 Error/System states (404, 500, Offline, 429 Rate Limit), 4 Milestone Badges, and 8 Custom Reaction Stickers. The remaining **186 Expansion Poses** are produced in Phase 12 using a modular vector rig where interchangeable head angles (3), eye/mouth expression layers (12), arm/prop rigs (15), and leaf crowns (6) snap onto standardized anchor coordinates and pass an automated CI SVG linter.

Google Stitch MCP Workflow and Screen-to-Code Mapping

In 2026, Google Stitch provides an AI-native UI generation platform equipped with a remote Model Context Protocol (MCP) server hosted directly by Google at https://stitch.googleapis.com/mcp. Coding agents such as Claude Code, Cursor, Antigravity, and Gemini CLI connect to this HTTP endpoint using the X-Goog-Api-Key header and invoke structured tools including create_project, list_projects, generate_screen_from_text, extract_design_context, fetch_screen_code, fetch_screen_image, list_screens, and get_screen. Furthermore, Stitch supports exporting a portable DESIGN.md specification containing semantic color tokens, typography scales, spacing rules, and component instructions that coding agents read automatically to preserve visual consistency across frontend and backend tasks.



Because Google Stitch's free tier provides a quota of 350 generations per month, generating dozens of screens before calibrating the design system wastes quota and creates visual drift. The engineering process enforces a ten-step closed-loop workflow. First, the team provisions a Stitch project (create_project named Swiftradish-Web-PWA) and configures the MCP client to connect to https://stitch.googleapis.com/mcp with the X-Goog-Api-Key header stored exclusively

in local untracked environment variables. Second, the team commits the canonical DESIGN.md file defining the Nocturnal Botanical Studio tokens into the repository root before generating any screens. Third, using Gemini 3 in Stitch Web mode, the team invokes generate_screen_from_text to generate only four representative anchor screens across Desktop and Mobile viewports: **SCR-11** (Home Feed), **SCR-13** (Post & Co-Plot Detail), **SCR-15** (Composer Modal), and **SCR-28** (Active Chat Canvas). Fourth, the team downloads the high-resolution rendered screenshots via fetch_screen_image and extracts the compiled style tokens via extract_design_context to inspect the actual visual output rather than trusting prompt text alone. Fifth, any layout anomalies are refined in Stitch until the four anchor screens pass visual review, locking SCR-11 as the master context source.
Sixth, the remaining 36 screens are generated in six manageable functional batches (Batch 1: Public & Auth SCR-01–07; Batch 2: Onboarding & PWA SCR-08–10; Batch 3: Creation & Co-Plots SCR-12, SCR-14, SCR-16–18; Batch 4: Discovery, Search & Patches SCR-19–22; Batch 5: Profiles, Bookmarks & Messaging SCR-23–30; Batch 6: Notifications, Settings, Safety & Errors SCR-31–40), always passing the extracted design context from SCR-11 into generate_screen_from_text. Seventh, after each batch, the agent executes list_screens and get_screen to log the real Stitch project_id and screen_id into docs/stitch-screen-ledger.md, strictly prohibiting any claim that a screen was generated unless its real ID and screenshot are archived. Eighth, desktop, mobile, and state variants (Loading, Empty, Populated, Error) are audited for completeness. Ninth, the agent calls fetch_screen_code to retrieve the raw HTML/Tailwind markup and refactors it into maintainable React Server and Client Components built on Radix UI and shadcn/ui primitives. **Crucial Engineering Boundary:** Stitch generates static visual layouts and styling tokens; it does not provide working ARIA focus traps, JWT session verification, database transactions, presigned R2 uploads, or WebSocket state synchronization, all of which must be wired in the application layer. Tenth, every implemented route undergoes a Playwright visual inspection against the Stitch screenshot at 1440\times 900 and 390\times 844 before completion.

Screen ID	Next.js Route	Stitch Project ID	Stitch Screen ID (Desktop / Mobile)	State Variants Verified (Load/Empty/Err)	Component Extraction (Radix + DESIGN.md)	Verification Status
SCR-11	/home	*Pending Batch 0*	*Pending list_screens*	[ ] Load [ ] Empty [ ] Err	FeedCard, CoPlotBanner, NavRail	UNVERIFIED (Pre-Generation)
SCR-15	/compose	*Pending Batch 0*	*Pending list_screens*	[ ] Empty [ ] Uploading [ ] Retry	ComposerModal, MediaTray, ProgressRing	UNVERIFIED (Pre-Generation)
SCR-28	/messages/[id]	*Pending Batch 0*	*Pending list_screens*	[ ] Sync [ ] Optimistic [ ] Offline	ChatVirtualList, MessageBubble, ReconnectPill	UNVERIFIED (Pre-Generation)

Document F: System Architecture, Data Model, and Media Pipeline

System Context, Container Topology, and Module Boundaries

Swiftradish is architected across four clearly separated infrastructure tiers that balance high concurrency with low operational complexity for a lean team using AI coding tools:

1. **Tier 1 — Client Layer (Browser & Installed PWA):** Runs the Next.js React client tree, mounting a persistent RealtimeWebSocketProvider in the root authenticated layout (app/(main)/layout.tsx) so socket connections survive client-side route transitions, paired with TanStack Query for client cache invalidation, Zustand for ephemeral composer/UI state, IndexedDB for draft and outbox persistence, and a Service Worker handling W3C Web Push notifications and offline bookmark caching.
2. **Tier 2 — Edge & Media Delivery Layer (Cloudflare):** Routes all public media reads through Cloudflare CDN backed by a **Cloudflare R2** object storage bucket (cdn.swiftradish.app) with zero egress fees, while isolating private direct-message attachments in a separate non-public R2 bucket accessed via short-lived HMAC-signed URLs. In Phase 12, short-form video transcoding and adaptive HLS delivery are handled by **Cloudflare Stream**.
3. **Tier 3 — Application Compute Layer (Next.js App Router + FastAPI 0.137+ Modular Monolith + arq Workers):** Next.js App Router handles initial Server-Side Rendering (SSR) and dynamic OpenGraph metadata generation. All business logic, REST APIs (/api/v1/*), and stateful WebSocket connections (/ws) run inside a modular **Python 3.12+ / FastAPI 0.137+ (Starlette 1.0+)** monolith using the lifespan context manager. Asynchronous jobs—including libvips image processing, Web Push fanout, and orphaned media cleanup—execute in separate **arq (Asyncio Redis Queue)** worker processes sharing the same codebase.
4. **Tier 4 — Data, State & Identity Layer (PostgreSQL 16 + Redis 7 + Supabase Auth):** **PostgreSQL 16** (accessed via SQLAlchemy 2.0 Async, asyncpg, and PgBouncer transaction pooling) serves as the single relational source of truth and powers full-text and fuzzy search via native tsvector and pg_trgm GIN indexes. **Redis 7** backs WebSocket Pub/Sub fanout, arq job queues, rate-limit counters, and JWT session revocation denylists. **Supabase Auth** manages identity registration, OAuth handshakes, and JWT issuance, while FastAPI verifies JWT signatures locally against cached JWKS keys. Nightly encrypted database backups are archived to **Backblaze B2** cold storage.

Architectural Decision Records (ADRs) for Core Stack Components

Decision Area	Recommended Choice	Credible Alternatives Considered	Evidence & Technical Justification	Revisit Trigger (When to Change)
ADR-01: Frontend Framework & Rendering	**Next.js App Router (React 19, TypeScript Strict, Tailwind + Radix/shadcn,**	1. Vite + React SPA 2. FastAPI + HTMX/Jinja2 3. Remix	Next.js App Router provides fast SSR + dynamic OpenGraph generation for	Only if Node SSR build overhead outweighs public SEO/unfurl needs.

Decision Area	Recommended Choice	Credible Alternatives Considered	Evidence & Technical Justification	Revisit Trigger (When to Change)
	TanStack Query, Zustand)		public post/Co-Plot deep links (/p/[id]), while Client Components + TanStack Query + Zustand power rich interactive composers and persistent WebSocket state. Strict downward layer imports (@app/features/* \rightarrow @app/components/* \rightarrow @app/shared/*) are enforced via eslint-plugin-boundaries.	
ADR-02: Backend API & Realtime Server	**Python 3.12+ with FastAPI 0.137+ / Starlette 1.0+ (Modular Monolith)**	1. Next.js Route Handlers only 2. Go (Chi/Fiber) 3. Node.js (NestJS)	Next.js serverless Route Handlers cannot hold open persistent WebSocket connections. FastAPI provides high-performance asyncio HTTP + native WebSocketRoute support, strict Pydantic v2 validation, and clean 4-file domain modules (_router.py, _servic[span_294](start_span)[span_294](end_span)e.py, _repository.py, _schemas.py). Uses Starlette 1.0's lifespan	If a specific CPU-bound hot path (e.g., custom ranking at >5,000 RPS) requires extracting a single Rust/Go worker.

Decision Area	Recommended Choice	Credible Alternatives Considered	Evidence & Technical Justification	Revisit Trigger (When to Change)
			context manager to manage async DB and Redis pools cleanly.	
ADR-03: Authentication & Session Verification	**Supabase Auth (Managed GoTrue) + Local JWKS/JWT Verification in FastAPI**	1. Custom Roll-Your-Own Auth 2. Auth0 / Clerk 3. Calling supabase.auth.get_user() on every request	Supabase Auth handles OAuth, OTPs, and refresh token rotation cheaply (@supabase/ssr sets HttpOnly cookies in Next.js). However, calling supabase.auth.get_user(token) over HTTP on every FastAPI request adds 40\text{ms}-120\text{ms} of network latency. Instead, FastAPI verifies the JWT signature locally in < 0.5\text{ms} using PyJWT against cached JWKS keys (audience="authenticated") and checks a Redis revocation denylist for logged-out sessions.	If MAU exceeds 500,000 and self-hosted Ory/Keycloak or native FastAPI OAuth becomes cheaper than managed auth tiers.
ADR-04: Primary Database & Search Engine	**PostgreSQL 16 (SQLAlchemy 2.0 Async + asyncpg + Alembic) with tsvector & pg_trgm**	1. Postgres + Elasticsearch 2. Postgres + Meilisearch/Typesense 3. MongoDB	PostgreSQL 16 handles relational integrity, keyset pagination, and transactional counters. Using a generated STORED tsvector column combined with pg_trgm GIN	When posts table exceeds 15M–20M active rows or write amplification on GIN indexes pushes p95 search latency above 100\text{ms}.

Decision Area	Recommended Choice	Credible Alternatives Considered	Evidence & Technical Justification	Revisit Trigger (When to Change)
			indexes provides sub-15ms weighted full-text and fuzzy trigram search up to millions of rows without syncing a separate search cluster.	
ADR-05: Background Jobs & Realtime Broker	**Redis 7 + arq (Asyncio Redis Queue)**	1. FastAPI BackgroundTasks 2. Celery + RabbitMQ 3. Temporal	FastAPI's built-in BackgroundTasks runs in the web server process with zero durability across restarts. Celery was designed for synchronous code and requires heavy process forking. Because Redis 7 is already required for WebSocket Pub/Sub fanout and rate limiting, arq reuses the exact same Redis instance to run native async def jobs with retries, cron scheduling, and pessimistic timeouts.	If workflow orchestration requires multi-day sagas or CPU-heavy distributed pipelines across 20+ worker nodes.
ADR-06: Media Object Storage & Delivery	**Cloudflare R2 + Cloudflare CDN (Phase 12 Video: Cloudflare Stream)**	1. AWS S3 + CloudFront 2. Supabase Storage 3. Backblaze B2	Social media workloads are overwhelmingly read/egress-heavy. AWS S3 charges $0.09/GB egress (4,600+/mo at 100TB). Cloudflare R2 charges **$0.00 egress**,	Never for public egress; R2's S3-compatible API guarantees zero vendor lock-in and $0 exit migration cost.

Decision Area	Recommended Choice	Credible Alternatives Considered	Evidence & Technical Justification	Revisit Trigger (When to Change)
			$0.015/GB/mo storage, and integrates natively with Cloudflare CDN so cached reads cost $0 in Class B operations. Glacier Deep Archive ($0.00099/GB) is used exclusively for cold encrypted PostgreSQL backups.	

Backend Domain Module Structure and Operational Contracts

FastAPI is partitioned into twelve internal domain modules under backe[span_243](start_span)[span_243](end_span)[span_248](start_span)[span_248](end_span)[span_253](start_span)[span_253](end_span)nd/src/modules/: identity, social_graph, posts, co_plots, media, feed, messaging, patches, search, notifications, achievements, and safety. Within each module, router.py parses HTTP or WebSocket payloads and invokes service.py; service.py enforces business logic and two-plane authorization (Plane 1 RBAC plus Plane 2 Object Ownership and Visibility) before calling repository.py; and repository.py executes typed SQLAlchemy 2.0 asyncpg queries. Cross-module mutations occur strictly through service interfaces or domain events, never by importing another module's repository directly. All endpoints are versioned under /api/v1/..., return standardized RFC 9457 Problem Details JSON on errors, and enforce PgBouncer transaction pooling with statement_timeout = 5000ms, lock_timeout = 2000ms, and idle_in_transaction_session_timeout = 10000ms.

Normalized PostgreSQL Data Model and Query Integrity

The PostgreSQL 16 schema separates normalized source-of-truth tables from explicitly managed counter columns and idempotency keys. Primary keys use **UUIDv7** (time-ordered UUIDs) so B-Tree index insertions are sequential and avoid the random page-splitting fragmentation of UUIDv4.

Entity / Table Name	Primary Key & Foreign Keys	Uniqueness & Integrity Constraints	Critical Indexes (B-Tree, GIN, Partial)	Retention, Deletion & Access Rules
users	**PK:** id (UUIDv7, matches Supabase auth.users.id)	UNIQUE (LOWER(username)), UNIQUE (LOWER(email)), CHECK (username ~ '^[a-z0-9_]{3,24}$'), CHECK (birth_year <= EXTRACT(YEAR FROM NOW()) - 18)	idx_users_username_trgm USING GIN (username gin_trgm_ops); idx_users_status (status, deleted_at)	Soft-deleted (deleted_at) for 14-day grace window, then scrubbed by arq purge job. Private fields (email, birth_year) never serialized to public DTOs.
user_profiles	**PK/FK:** user_id \rightarrow users(id) ON DELETE CASCADE | FK avatar_media_id \rightarrow media_assets(id) ON DELETE SET NULL, CHECK (char_length(bio) <= 300)	idx_profiles_search_vec USING GIN (search_vector)	Stores display_name, bio, theme_token, is_private, dm_policy, and

denormalized follower_count, following_count. | | **user_relationships** | **PK:** id (UUIDv7) **FKs:** actor_id, target_id \rightarrow users(id) ON DELETE CASCADE | UNIQUE (actor_id, target_id), CHECK (actor_id <> target_id), status IN ('PENDING', 'ACCEPTED') | idx_rel_target_status (target_id, status, created_at DESC); idx_rel_mutual (actor_id, target_id) WHERE is_mutual = TRUE | Database trigger updates is_mutual = TRUE on both directional rows when a reciprocal ACCEPTED follow is inserted, and decrements counters atomically on DELETE. | | **user_blocks & user_mutes** | **PK:** (blocker_id, blocked_id) **FKs:** \rightarrow users(id) ON DELETE CASCADE | PRIMARY KEY (blocker_id, blocked_id), CHECK (blocker_id <> blocked_id) | idx_blocks_reverse (blocked_id, blocker_id) | Checked via fast NOT EXISTS subquery (and cached in Redis per user session) on all feed, search, profile, and DM queries. | | **patches & patch_members** | **PK:** id (UUIDv7) **FK:** creator_id \rightarrow users(id) | UNIQUE (LOWER(slug)), UNIQUE (patch_id, user_id) in patch_members | idx_patches_trgm USING GIN (name gin_trgm_ops, description gin_trgm_ops) | Community spaces; deleting creator reassigns stewardship to senior moderator rather than cascading deletion of community posts. | | **posts** | **PK:** id (UUIDv7) **FKs:** author_id \rightarrow users(id) ON DELETE CASCADE, patch_id \rightarrow patches(id) ON DELETE SET NULL, remix_of_id \rightarrow posts(id) ON DELETE SET NULL | visibility IN ('PUBLIC', 'FOLLOWERS', 'MUTUALS', 'PATCH_ONLY'), post_type IN ('STANDARD', 'CO_PLOT') | idx_posts_author_cursor (author_id, id DESC) WHERE deleted_at IS NULL; idx_posts_patch_cursor (patch_id, id DESC); idx_posts_fts USING GIN (search_vector) | Includes body, search_vector (GENERATED ALWAYS AS ... STORED), and denormalized reaction_count, comment_count, remix_count. Soft-delete sets deleted_at and enqueues R2 media purge. | | **co_plots & co[span_11](start_span)[span_11](end_span)_plot_participants** | **PK/FK:** post_id \rightarrow posts(id) ON DELETE CASCADE **FK:** conversation_id \rightarrow conversations(id) | UNIQUE (post_id, user_id) in co_plot_participants, rsvp_status IN ('GOING', 'MAYBE', 'DECLINED'), CHECK (capacity IS NULL OR capacity > 0) | idx_coplots_starts_at (starts_at ASC) WHERE status = 'ACTIVE'; idx_coplot_user (user_id, rsvp_status) | Links a post directly to structured coordination metadata (title, starts_at, location_label, allow_carousel_contributions) and its companion chat room. | | **media_assets** | **PK:** id (UUIDv7) **FKs:** owner_id \rightarrow users(id) ON DELETE CASCADE, post_id \rightarrow posts(id) ON DELETE CASCADE, message_id \rightarrow messages(id) ON DELETE CASCADE | UNIQUE (r2_object_key), status IN ('PENDING_UPLOAD', 'PROCESSING', 'READY', 'FAILED', 'QUARANTINED'), CHECK (char_length(alt_text) <= 1000) | idx_media_post_order (post_id, position ASC) WHERE status = 'READY'; idx_media_orphan_cleanup (status, created_at) WHERE post_id IS NULL AND message_id IS NULL | Prevents orphaned media: any asset still in PENDING_UPLOAD or unattached (post_id IS NULL AND message_id IS NULL) after 6 hours is deleted from R2 and Postgres by arq cleanup cron. | | **post_reactions & bookmarks** | **PK:** id (UUIDv7) **FKs:** post_id \rightarrow posts(id) ON DELETE CASCADE, user_id \rightarrow users(id) ON DELETE CASCADE | UNIQUE (post_id, user_id, reaction_code) for reactions; UNIQUE (user_id, post_id) for bookmarks | idx_reactions_post (post_id, reaction_code); idx_bookmarks_user_cursor (user_id, collection_id, id DESC) | Strict UNIQUE constraint + INSERT ... ON CONFLICT DO NOTHING makes duplicate reactions mathematically impossible even under concurrent double-taps. | | **comments** | *PK:* id (UUIDv7) **FKs:** post_id \rightarrow posts(id)[span_12](start_span)[span_12](end_span) ON DELETE CASCADE, author_id \rightarrow users(id) ON DELETE C[span_13](start_span)[span_13](end_span)ASCADE, parent_id \rightarrow comments(id) ON DELETE CASCADE | CHECK (depth BETWEEN 0 AND 2) (enforces max 3 nesting levels) | idx_comments_post_tree (post_id, parent_id, id ASC) | If a parent comment with child replies is deleted by its author, body is scrubbed to NULL and

is_tombstone = TRUE so child discourse remains intact. | | **conversations & conversation_participants** | **PK:** id (UUIDv7) **FKs:** conversation_id, user_id | UNIQUE (conversation_id, user_id), UNIQUE (direct_pair_key) (for 1:1 chats: LEAST(u1,u2) \|\| ':' \|\| GREATEST(u1,u2)) | idx_conv_part_user_sort (user_id, is_pinned DESC, last_message_at DESC) | Stores last_read_message_id (UUIDv7) per participant rather than a mutable integer counter, completely eliminating unread-count race conditions. | | **messages** | **PK:** id (UUIDv7) **FKs:** conversation_id \rightarrow conversations(id) ON DELETE CASCADE, sender_id \rightarrow users(id) ON DELETE SET NULL | UNIQUE (conversation_id, sender_id, client_msg_id) | idx_messages_conv_cursor (conversation_id, id DESC); idx_messages_fts USING GIN (to_tsvector('simple', coalesce(body, ''))) | UNIQUE (conversation_id, sender_id, client_msg_id) guarantees idempotent message creation across WebSocket retries. | | **notifications** | **PK:** id (UUIDv7) **FK:** recipient_id \rightarrow users(id) ON DELETE CASCADE | UNIQUE (recipient_id, dedupe_key) | idx_notif_recipient_unread (recipient_id, id DESC) WHERE read_at IS NULL | dedupe_key (e.g., react:post_123:2026-10-14) rolls up high-frequency reactions into a single updated row (actor_count = actor_count + 1). | | **safety_reports & audit_events** | **PK:** id (UUIDv7) **FKs:** reporter_id, target_user_id | UNIQUE (ticket_number), sla_tier IN ('CRITICAL_24H', 'STANDARD_72H', 'GENERAL_15D') | idx_reports_sla_queue (status, sla_deadline_at ASC) | audit_events is append-only (PostgreSQL trigger blocks UPDATE/DELETE), retaining statutory moderation evidence for 180 days per India IT Rules. |

This relational design eliminates nine classic failure modes in social platforms. Duplicate reactions, duplicate follow edges, and double-submitted messages are blocked at the storage engine level via composite UNIQUE constraints paired with ON CONFLICT DO NOTHING. Unread message count race conditions are eliminated by recording last_read_message_id (UUIDv7) per participant and comparing it against conversations.last_message_id via an index-only scan rather than incrementing a mutable integer column. Orphaned media objects are prevented by an arq cron job that queries idx_media_orphan_cleanup every hour and deletes any R2 object and database row where post_id IS NULL AND message_id IS NULL for more than six hours. Unbounded feed queries are prevented by banning SQL OFFSET across the codebase in favor of keyset cursor pagination (WHERE id < :cursor_uuidv7 ORDER BY id DESC LIMIT 25). N+1 query regressions are prevented by configuring SQLAlchemy models with lazy="raise" and loading associations via explicit selectinload() batch queries. Finally, unified search combines semantic stemming (tsvector) with typo-tolerant trigram matching (pg_trgm) using the weighted ranking expression:

$$\text{Rank} = 2.0 \cdot \text{ts_rank}(\text{search_vector}, \text{websearch_to_tsquery}([span_340](start_span)[span_340](end_span)\text{query})) + 1.0 \cdot \text{similarity}(\text{title}, \text{query}) + 0.5 \cdot \text{word_similarity}(\text{query}, \text{body})$$

Direct-to-Storage Media Lifecycle and Video Engineering

Proxying multi-image carousels or video uploads through FastAPI workers saturates application memory, inflates API tail latency, and doubles cloud bandwidth charges. Swiftradish implements **Direct-to-Storage Presigned Uploads with Cryptographic Server Finalization**. When a user selects files in the composer, the client calls POST /api/v1/media/upload-intents with {filename, content_type, byte_size}. FastAPI validates per-user rate limits (maximum 30 media uploads per hour, 10 files per post,
15$$\text{[span_31](start_span)[span_31](end_span)[span_45](start_span)[span_45](end_span)M

B} per image), inserts media_assets rows in PENDING_UPLOAD status, and returns S3-compatible presigned PUT URLs targeting a staging prefix in Cloudflare R2 (staging/{user_id}/{media_id}) with strict Content-Length and Content-Type conditions signed into the 15-minute URL. Once the browser streams the file directly to R2 and invokes POST /api/v1/media/{id}/complete, FastAPI executes an S3 HeadObjec[span_331](start_span)[span_331](end_span)[span_333](start_span)[span_333](end_span)t and a Range: bytes=0-4095 read against R2 to verify that the object exists, matches the authorized byte length, and contains valid magic file signatures (JPEG, PNG, WebP, or AVIF), preventing clients from claiming nonexistent or spoofed objects were uploaded.
Validated images are processed via pyvips (libvips C library running inside asyncio.to_thread()), which decodes images in streaming chunks using under 30\text{MB} of RAM, rejects decompression bombs exceeding 64 megapixels, strips all EXIF/IPTC/GPS location metadata, computes a 32-character Blurhash string, writes optimized WebP variants (thumb_400w, feed_1080w, full_20[span_311](start_span)[span_311](end_span)48w) to the public delivery prefix (media/{media_id}/{variant}.webp), deletes the staging object, and marks status = 'READY'. Public assets are served through cdn.swiftradish.app with Cache-Control: public, max-age=31536000, immutable so cached edge reads cost 0 in R2 Class B operations. Private DM attachments are stored in an isolated private R2 bucket and accessed exclusively via 15-minute signed URLs after verifying conversation membership. For Phase 12 short-form video (\le 60\text{s}$), rather than storing raw MP4 files or operating self-managed FFmpeg clusters, Swiftradish integrates **Cloudflare Stream** ($5 per 1,000 minutes stored plus $1 per 1,000 minutes delivered, with zero egress fees), gaining resumable tus uploads, automatic multi-bitrate HLS/DASH transcoding, thumbnail generation, and adaptive playback.

Realtime Messaging, Presence, and Notification Delivery

Because Next.js App Router unmounts page components during client-side navigation, the RealtimeWebSocketProvider is mounted at the root authenticated layout (src/app/(main)/layout.tsx) to maintain a single persistent connection across route changes. To authenticate the WebSocket without exposing long-lived JWTs in URL query strings, the client requests a single-use, 30-second random nonce via POST /api/v1/ws/ticket (SETEX ws_ticket:{nonce} 30 {user_id} in Redis) and opens wss://api.swiftradish.app/ws?ticket={nonce}.
Swiftradish explicitly guarantees **At-Least-Once Delivery with Idempotent Deduplication and Per-Conversation Causal Ordering**; it avoids false claims of network-level "exactly-once" transport by combining client retry queues, PostgreSQL's UNIQUE (conversation_id, sender_id, clie[span_183](start_span)[span_183](end_span)[span_186](start_span)[span_186](end_span)[span_189](start_span)[span_189](end_span)nt_msg_id) constraint, and client-side state deduplication by server_msg_id. To avoid O(N^2) presence broadcast storms, online presence and typing indicators (throttled to one pulse per 3 seconds) are scoped strictly to open conversations and active Co-Plot rooms.
For durable notifications and PWA Web Push, FastAPI writes deduplicated notifications rows (ON CONFLICT (recipient_id, dedupe_key) DO UPDATE) and enqueues an arq job. If the recipient is not actively viewing the conversation and is outside their configured Quiet Hours (SCR-35), the worker dispatches W3C Web Push payloads signed with VAPID keys. In accordance with Apple's iOS 16.4+ and iOS 19 Web Push standards, push subscriptions are requested only after the user installs the PWA to their Home Screen (display: standalone) and taps an explicit in-app opt-in trigger. Furthermore, the Service Worker's push event listener parses payloads defensively, always invokes self.registration.showNotification() (as iOS forbids

silent background pushes), and scopes the notification tag to the specific conversation_id or post_id rather than a generic static tag so unrelated notifications never overwrite one another.

Document G: Reliability, Security, Performance, and Operational Economics

Threat Model, Object-Level Authorization, and Regulatory Compliance

Threat Vector	Attack Mechanism	Architectural Control & Mitigation	Verification Test
Broken Object-Level Authorization (BOLA / IDOR)	Attacker substitutes another user's post_id, conversation_id, or media_id in API calls.	Every service method pairs Plane 1 (RBAC) with Plane 2 (Object Policy). Private resources return 404 Not Found (not 403 Forbidden) to unauthorized callers to prevent existence leakage.	Automated matrix test attempting cross-user read/edit/delete across all 14 entities.
Account Enumeration & Credential Stuffing	Automated bots probe /register, /login, and /auth/reset-password to harvest valid emails or brute-force passwords.	Identical response timing and copy on password reset; Redis sliding-window rate limiter (5\text{ attempts}/15\text{min} per IP and per account) + Cloudflare Turnstile challenge on suspicious velocity.	Integration test verifying identical HTTP status and payload shape on known vs unknown emails.
Malicious File Upload, Polyglots & Decompression Bombs	Attacker uploads an HTML/SVG script disguised as .jpg or a 2\text{MB} PNG that expands to 20\text{GB} in RAM (pixel flood).	SVG uploads by end-users are completely banned; libvi[span_298](start_span)[span_298](end_span)ps enforces max 8192 \times 8192\text{px} dimensions before decoding and re-encodes every pixel buffer into a fresh WebP/AVIF container; cdn.swiftradish.app serves all assets with Content-Security-Policy	Upload test suite with EICAR, SVG-in-JPEG polyglots, and 100,000 \times 100,000\text{px[span_122](start_span)[span_122](end_span)} header bombs.

Threat Vector	Attack Mechanism	Architectural Control & Mitigation	Verification Test
		: default-src 'none', X-Content-Type-Options: nosniff, and Content-Disposition: inline.	
Stored XSS & Unsafe Rich Content	Attacker injects javascript: URLs or HTML tags inside Markdown posts, bios, or chat messages.	dangerouslySetInnerHTML is banned in ESLint; Markdown AST is rendered into safe React nodes with URL protocol whitelisting (https://, mailto: only).	Fuzz test with OWASP XSS payload dictionary across all text inputs.
Server-Side Request Forgery (SSRF)	Link preview fetcher or webhook tricked into querying cloud metadata IPs (169.254.169.254) or 127.0.0.1 (localhost).	Link metadata worker uses a dedicated outbound proxy that resolves DNS first and blocks all RFC 1918 private, loopback, and link-local IPv4/IPv6 ranges, enforces a 2\text{MB} response cap, and times out at 2500\text{ms}.	Unit and integration tests targeting loopback, 169.254.169.254, and DNS rebinding mocks.
Private Media Leakage	Attacker guesses or shares URLs for private DM attachments or deleted posts.	Private DM attachments live in a non-public R2 bucket requiring HMAC-signed URLs with a 15\text{min} TTL; deleting a post/message immediately enqueues an arq job that executes DeleteObject on R2 and purges the Cloudflare CDN cache URL.	Verify signed URL expires after 900\text{s} and deleted post media returns 404 after purge job.

Legal and Regulatory Compliance (India Launch Geography and Global Baseline)

Because Swiftradish launches with user coverage in India, the platform architecture embeds the statutory due-diligence requirements of the **Information Technology (Intermediary Guidelines and Digital Media Ethics Code) Rules, 2021 (as amended)** and the **Digital Personal Data Protection (DPDP) Act, 2023** directly into its database schema and moderation workflows. Under Rule 3 of the IT Rules 2021, Swiftradish prominently publishes the name and contact details of its Grievance Officer on /legal/grievance and inside /settings/reports (SCR-38). Every

user report generates a unique tracking ticket number and dispatches an automated acknowledgment within 60 seconds, well within the statutory **24-hour acknowledgment** ceiling. Complaints regarding non-consensual intimate imagery, invasions of bodily privacy, or identity morphing and impersonation are routed into the CRITICAL_24H priority queue and automatically placed under a temporary quarantine shield so access is disabled within the **24-hour removal** window. Court or law enforcement information orders and general unlawful content reports are tracked under a **72-hour** SLA clock, while general user grievances are resolved within **15 days**. In accordance with Rule 3(1) and Rule 4(8), users whose content is disabled receive formal notice and an in-app dispute workflow (SCR-38), along with explicit instructions on their right to appeal the Grievance Officer's decision to the government-established **Grievance Appellate Committee (gac.gov.in)** within **30 days**. Under Section 9 of the DPDP Act 2023, Swiftradish enforces an 18+ age gate at registration to avoid collecting personal data from minors without verifiable parental consent. Before scaling commercially, qualified legal counsel must review three operational thresholds: (a) appointing a Resident Grievance Officer, Chief Compliance Officer, and 24x7 law enforcement Nodal Contact Person if Indian user registrations cross the 5 million "Significant Social Media Intermediary" (SSMI) threshold; (b) publishing Terms and Privacy Policies across Eighth Schedule Indian languages; and (c) adhering to CERT-In 6-hour cybersecurity incident reporting rules.

Performance Budgets, Caching Strategy, and Scaling Triggers

Metric / Operation	Target Budget (Mid-Tier Mobile / 4G Network)	Architectural Enforcement Mechanism
Initial Page Load (LCP / INP / CLS)	\text{LCP} \le 1.8\text{s}, \text{INP} \le 120\text{ms}, \text{CLS} \le 0.03	Next.js Server Components for initial shell; explicit width/height + Blurhash placeholders on every media slot so layout never shifts; initial JS bundle capped at < 140\text{KB} gzipped.
Route Transitions & Feed Pagination	Route switch \le 100\text{ms} perceived; cursor fetch p95 \le 90\text{ms}	Prefetched App Router links; TanStack Query infinite scroll pre-fetches next 20 items when scroll reaches 75\% of viewport; virtualized DOM lists (@tanstack/react-virtual) when feed/chat exceeds 60 items.
Image Loading & Video Startup	Thumbnail visible in \le 200\text{ms}; HLS video first frame \le 800\text{ms}	Responsive <img srcSet> serving 400w/1080w[span_198](start_span)[span_198](end_span)[span_205](start_span)[span_205](end_span) WebP from Cloudflare CDN edge PoPs; max 1 active video decoder playing in viewport at a time (IntersectionObserver pauses

Metric / Operation	Target Budget (Mid-Tier Mobile / 4G Network)	Architectural Enforcement Mechanism
		offscreen videos).
Chat Delivery & API Latency	WS message fanout p95 \le 120\text{ms}; REST API p95 \le 80\tex[span_33](start_span)[span_33](end_span)[span_47](start_span)[span_47](end_span)t{ms} (reads) / \le 150\text{ms} (writes)	Optimistic UI updates in < 16\text{ms} (1 frame); SQL queries budgeted to \le 15\text{ms} p95 via composite keyset indexes and EXPLAIN (ANALYZE, BUFFERS) CI checks.
Client Memory & Mobile Data Budget	Heap memory \le 120\text{MB} after 15 mins of scrolling; < 8\text{MB} per 50 feed cards	Virtualization unmounts off-screen high-res image textures; Service Worker caches static assets and icons aggressively.

Rather than over-provisioning distributed clusters prematurely, infrastructure evolves strictly when measured telemetry crosses three explicit scaling triggers. First, for **Feed Materialization**, at launch (< 25,000 DAU) GET /api/[span_286](start_span)[span_286](end_span)v1/feed/follow[span_312](start_span)[span_312](end_span)ing executes an indexed PostgreSQL keyset query joining posts against followed target_ids and patch_ids (< 12\text{ms}); only when posts exceeds 5 million rows or p95 feed query latency exceeds 60\text{ms} does the system enable a hybrid push/pull cache in Redis Sorted Sets (ZADD feed:{user_id} {timestamp} {post_id}, capped at 800 items) populated by arq workers for standard accounts while merging high-follower accounts on read. Second, for **Search Indexing**, unified search remains 100% inside PostgreSQL tsvector and pg_trgm GIN indexes until active posts exceed 15 million rows or GIN write amplification pushes p95 search latency above 100\text{ms}. Third, for **WebSocket Gateways**, a single 2-vCPU / 4GB RAM FastAPI container handles 8,000 to 12,000 concurrent sockets; when active connections exceed 6,000 per node, horizontal container autoscaling triggers automatically behind Redis Pub/Sub sharded channels.

Quality Engineering, Schema Preflight Gates, and Testing Strategy

A frequent failure mode in AI-assisted development occurs when automated tests pass against an ephemeral in-memory schema created via Base.metadata.create_all(), while the actual development or production database crashes at runtime due to a missing or out-of-order migration. Swiftradish eliminates this class of bug through two non-negotiable architectural rules. First, Base.metadata.create_all() is banned across both application code and test suites; every test database is provisioned by running alembic upgrade head from zero, and FastAPI's lifespan startup hook executes a **Schema Compatibility Preflight Check** that queries SELECT version_num FROM alembic_version, compares it against the codebase's expected HEAD_REVISION, verifies pg_trgm installation, and halts server startup immediately if any migration drift exists. Second, production and application components (src/) are forbidden from importing mock data or silently falling back to fabricated posts; development seed scripts reside strictly in backend/scripts/seed_dev_fixtures.py and tag seeded rows with an explicit is_dev_fixture = TRUE banner that is blocked by a CHECK constraint in production.

Verification Gate	Tooling & Execution Scope	Mandatory Coverage Criteria	Release Gate Status Tag
1. Automated Unit & Schema Preflight	pytest, vitest, ruff, mypy --strict, tsc --noEmit, alembic upgrade head && alembic downgrade -1 && alembic upgrade head	100% of domain services, ranking math, Zod/Pydantic validators, and reversible migrations tested.	AUTOMATED-TESTED
2. Real-DB & Storage Integration	pytest-asyncio + Testcontainers (PostgreSQL 16, Redis 7, MinIO/R2 S3 mock)	Tests full JWT auth verification, BOLA/IDOR matrix across all endpoints, presigned upload + magic-byte check, Orphan media cleanup cron, and WebSocket message deduplication.	INTEGRATION-VERIFIED
3. Browser End-to-End & Viewport Matrix	Playwright across Desktop Chromium (1440\times 900) and Mobile WebKit/iOS (390\times 844) + @axe-core/playwright	Verifies all 15 mandatory user flows (Sign-up \rightarrow Multi-image upload \rightarrow Composer state reset \rightarrow Co-Plot join \rightarrow WS disconnect/reconnect \rightarrow Report/Block \rightarrow Account delete) with zero WCAG AA violations.	BROWSER-VERIFIED
4. Performance & Load Verification	Lighthouse CI + k6 HTTP/WebSocket load scripts	Verifies LCP < 1.8\text{s}, 500 concurrent WebSocket clients exchanging messages with p95 < 120\text{ms}, and zero N+1 SQL queries via query-count assertions.	PERFORMANCE-VERIFIED
5. Security & Compliance Audit	OWASP ZAP baseline + SAST (bandit/semgrep) + statutory SLA workflow check	Verifies polyglot upload rejection, SSRF IP blocking, expired JWT rejection, and 24h grievance ticket generation.	SECURITY-REVIEWED

Infrastructure Cost Scenarios and Operational Runbooks

The financial model below uses verified 2026 published pricing across Cloudflare R2 ($0.015/GB monthly storage, $0.00 egress, $4.50 per million Class A writes, $0.36 per million Class B reads, with a recurring free tier of 10 GB storage, 1 M Class A writes, and 10 M Class B reads), B 0.006/GB to $0.00695/GB$ cold storage with free egress up to 3x stored data), AWS S3 (0.023/GB storage plus $0.09/GB$ internet egress), and Cloudflare Stream (5 per 1,000 minutes stored plus $1 per 1,000 minutes delivered).

Cost Driver / Infrastructure Layer	Scenario 1: Seed Launch (1,000 MAU / ~250 DAU)	Scenario 2: Growing Community (25,000 MAU / ~7,500 DAU)	Scenario 3: Scaled Platform (250,000 MAU / ~75,000 DAU)	Controlling Assumptions & Operational Lever
Frontend Hosting (Next.js App Router)	$0 – $20/mo (Vercel Pro or single container)	$40/mo (2x Node SSR containers behind Cloudflare)	$180/mo (Auto-scaled container group)	Caching public routes and serving all static/media assets via Cloudflare CDN keeps SSR CPU minimal.
Backend API, WebSockets & arq Workers (FastAPI)	$14/mo (2x shared-CPU 1GB containers on Fly.io/Railway)	$85/mo (3x 2-vCPU/4GB API+WS nodes + 2x arq image workers)	$480/mo (10x API+WS nodes + 6x arq workers)	libvips processes images in streaming memory without needing GPU or huge RAM instances.
PostgreSQL 16 + PgBouncer + Backups	$0 – $25/mo (Supabase Pro / Neon Launch, 15GB storage)	$95/mo (Dedicated 4-vCPU / 16GB RAM + 150GB NVMe + PITR)	$520/mo (8-vCPU / 32GB Primary + 1 Read Replica + 1TB storage)	UUIDv7 sequential keys + keyset cursors prevent IOPS bloat; nightly encrypted pg_dump archived to Backblaze B2 at $0.006/GB.
Authentication (Supabase Auth)	Included in $25/mo Pro base (up to 100,000 MAU)	Included in $25/mo base (<100k MAU)	~$487/mo (100k included + 150k \times $0.00325/MAU)	Local JWKS verification in FastAPI imposes zero API overhead on Auth servers.
Media Storage & Egress (Cloudflare R2 + CDN)	**$0.08/mo** (15GB stored, 50k writes, 2M uncached reads, 500GB egress = $0)	**$22.65/mo** (500GB stored, 2M writes, 40M reads, 5TB egress = $0)	**$245.00/mo** (8TB stored = $120, 15M writes = $67.50, 160M uncached reads = $57.60, 80TB egress = **$0**)	Compare Scenario 3 against AWS S3, where 80TB of egress alone would cost **$7,200/month** ($0.09/GB). Long Cache-C[span_133](start_span)[span_133](end_span)

Cost Driver / Infrastructure Layer	Scenario 1: Seed Launch (1,000 MAU / ~250 DAU)	Scenario 2: Growing Community (25,000 MAU / ~7,500 DAU)	Scenario 3: Scaled Platform (250,000 MAU / ~75,000 DAU)	Controlling Assumptions & Operational Lever
				ontrol: immutable headers absorb 85%+ of reads at the CDN edge for $0.
Phase 12 Short Video (Cloudflare Stream, Optional)	$0 (Disabled in MVP; enabled post-launch)	$65/mo (5,000 mins stored = $25 + 40,000 mins viewed = $40)	$650/mo (50,000 mins stored = $250 + 400,000 mins viewed = $400)	Controlled strictly via 60s max clip length and daily creator upload caps.
Redis 7, Transactional Email & Observability	$10/mo (Managed Redis + Resend Email + Sentry free tier)	$55/mo (4GB Redis HA + Resend Pro + Sentry/Grafana Cloud)	$260/mo (16GB Redis cluster + transactional email + log retention)	Web Push is 100% free via native W3C VAPID protocol; email is used only for OTPs, password resets, and critical digests.
Estimated Total Monthly Infrastructure Run-Rate	**~$49 – $69 / month**	**~$363 / month** ($298 without video)	**~$2,822 / month** ($2,172 without video)	Achieves unit economics of **~$0.011 per MAU per month** at 250,000 MAU.

Accessibility, Internationalization, PWA Standards, and Operational Runbooks

Swiftradish's Web App Manifest (manifest.webmanifest) is served from the root with display: "standalone", providing 192x192 and 512x512 PNG icons alongside dedicated "purpose": "maskable" icons whose artwork stays strictly within the inner 80% (409x409 circle) safe zone so Android adaptive launchers never clip the logo, paired with a 180x180 apple-touch-icon.png for iOS Safari. All text columns store UTF-8 (NFC-normalized) Unicode supporting Indic scripts, RTL layouts (dir="auto"), and grapheme-cluster-aware truncation (Intl.Segmenter), while all timestamps are persisted in TIMESTAMPTZ (UTC) and formatted client-side via Intl.DateTimeFormat. Shared-device safety ensures that logging out immediately clears HttpOnly cookies, purges IndexedDB drafts/caches, unregisters the active Service Worker push endpoint, and adds the JWT's jti to the Redis revocation list. Operational resilience relies on continuous PostgreSQL WAL archiving (RPO < 5\text{ minutes}) plus nightly encrypted logical backups pushed to an air-gapped Backblaze B2 bucket with Object Lock enabled, verified monthly via an automated restore drill into an ephemeral staging container (RTO < 30\text{ minutes}).

Document H: Dependency-Aware Implementation

Roadmap

Phased Execution Plan and Verification Contracts

The implementation is structured into **12 dependency-ordered phases** designed for a developer or lean team operating with AI coding agents. Visual Design System and Stitch Screen Catalog generation (Phase 1) and Database Migrations and Auth Foundations (Phases 2–3) precede feature coding, while Media Upload Infrastructure (Phase 4) precedes Posts and Co-Plots (Phase 5) because posts depend on verified media asset records.

Phase	Objective & Prerequisites	Key Modules & Database Changes	Core API Contracts & Screens	Mandatory Tests & Completion Evidence	Rollback & Risk Mitigation
Phase 1: Design System, DESIGN.md & Stitch Catalog	Establish visual tokens, 30 core Pip SVG poses, and generate verified Stitch screens. *Prereq: Product sign-off.*	DESIGN.md, frontend/src/styles/globals.css, public/pip/core/*.svg, docs/stitch-screen-ledger.md.	Stitch MCP calls (create_project, generate_screen_from_text, extract_design_context, fetch_screen_image).	Screenshot archive + populated Stitch Screen IDs for all anchor and batch screens; contrast audit (\ge 4.5:1).	Regenerate non-compliant screens using locked SCR-11 design context.
Phase 2: Repo Scaffolding, DB Preflight & CI Gate	Boot monorepo (frontend/, backend/), Docker Compose (Postgres 16, Redis 7, MinIO), Alembic preflight check, and CI. *Prereq: Phase 1.*	backend/src/core/lifespan.py, backend/alembic/versions/0001_init_extensions.py (pg_trgm, UUIDv7).	GET /api/v1/health (checks DB revision match + Redis ping with 500\text{ms} budget). SCR-40 error shells.	CI passes alembic upgrade head, al[span_361](start_span)[span_361](end_span)[span_364](start_span)[span_364](end_span)embic downgrade base, and verifies server refuses to start on schema mismatch.	Standard git revert; zero production data exists yet.
Phase 3: Auth, Sessions, Profiles & Hybrid Social Graph	Implement 18+ registration, Supabase JWT/JWKS local verification, profiles, and follow/mutual graph. *Prereq:*	Tables: users, user_profiles, user_relationships. Modules: identity, social_graph.	POST /api/v1/auth/register, GET/PATCH /api/v1/users/me, POST /api/v1/relations/{userId}/follow. Screens:	Integration tests for <18 DOB rejection, expired/tampered JWT 401 rejection, mutual trigger accuracy, and private profile	Reversible Alembic migration 0002_identity_graph.[span_458](start_span)[span_458](end_span)py; feature-flag

Phase	Objective & Prerequisites	Key Modules & Database Changes	Core API Contracts & Screens	Mandatory Tests & Completion Evidence	Rollback & Risk Mitigation
	Phase 2.		SCR-01, SCR-04 to SCR-09, SCR-23 to SCR-25.	404 masking.	OAuth providers.
Phase 4: Direct-to-R2 Media Pipeline & Composer State Machine	Build presigned R2 uploads, libvips EXIF stripping/WebP resizing, orphan cleanup cron, and deterministic composer reset. *Prereq: Phase 3.*	Table: media_assets. Module: media + arq worker (process_image, purge_orphan_media).	POST /api/v1/media/upload-intents, POST /api/v1/media/{id}/c[span_300](start_span)[span_300](end_span)omplete, DELETE /api/v1/media/{id}. Screens: SCR-14 to SCR-16.	E2E Playwright test verifying: single/multi-image upload \rightarrow progress ring \rightarrow success \rightarrow composer state reset \rightarrow second immediate upload; plus polyglot/EXIF strip test.	Abort incomplete multipart uploads; fallback to synchronous libvips resize if worker lags.
Phase 5: Core Posts, Carousels & Co-Plot Coordination	Enable publishing standard posts and interactive Co-Plots (RSVPs + collaborative carousels). *Prereq: Phase 4.*	Tables: posts, co_plots, co_plot_participants, bookmarks. Modules: posts, co_plots.	POST /api/v1/posts, GET/DELETE /api/v1/posts/{id}, POST /api/v1/plots/{id}/rsvp, POST /api/v1/plots/{id}/submissions. Screens: SCR-02, SCR[span_301](start_span)[span_301](end_span)-13, SCR-17, SCR-18, SCR-26.	E2E test: Host creates Co-Plot \rightarrow Mutual RSVPs & submits photo \rightarrow Host approves \rightarrow slide appears in carousel; BOLA test blocking non-mutuals.	Reversible migration 0004_posts_coplots.py; disable collaborative slide uploads via config flag if abused.
Phase 6: Dual-Mode Feeds,	Ship chronological ("My Patch")	Tables: post_reactions, comments.	GET /api/v1/feed/following, GET	Keyset pagination test verifying zero	Switch discovery feed to simple

Phase	Objective & Prerequisites	Key Modules & Database Changes	Core API Contracts & Screens	Mandatory Tests & Completion Evidence	Rollback & Risk Mitigation
Reactions, Comments & Remixes	and transparent discovery ("Fresh Harvest") feeds, reactions, and 3-level comments. *Prereq: Phase 5.*	Module: feed.	/api/v1/feed/discover, POST /api/v1/posts/{id}/reactions, POST /api/v1/posts/{id}/comments. Screens: SCR-11, SCR-12.	duplicate posts across pages when new posts arrive; concurrent reaction idempotency test; zero N+1 query check.	chronological public stream if ranking query latency spikes.
Phase 7: Radish Patches (Communities) & Hybrid Search	Launch topic spaces (Patches) and unified tsvector + pg_trgm search across users, posts, and patches. *Prereq: Phase 6.*	Tables: patches, patch_members, GIN indexes on search_vector and trigram columns. Modules: patches, search.	POST/GET /api/v1/patches, GET /api/v1/search?q=.... Screens: SCR-03, SCR-19 to SCR-22.	Search relevance & fuzzy typo test ("urbn sktching" matches "Urban Sketching" via pg_trgm); abort-controller test on rapid typing.	Adjust pg_trgm.similarity_threshold dynamically; disable fuzzy fallback if CPU spikes.
Phase 8: Real-Time Messaging & Co-Plot Chat Rooms	Implement WebSocket gateway, Redis Pub/Sub fanout, 1:1/group/Co-Plot chats, message requests, and gap sync. *Prereq: Phase 5, 7.*	Tables: conversations, conversation_participants, messages. Module: messaging.	POST /api/v1/ws/ticket, WS /ws, GET /api/v1/conversations/{id}/messages. Screens: SCR-27 to SCR-30.	Playwright network-drop simulation test: drop WS \rightarrow queue message \rightarrow reconnect \rightarrow verify sync.resume delivers missed messages with zero duplicates.	Degrade gracefully to REST POST + 10s polling if WebSocket cluster experiences instability.
Phase 9: Activity Notifications, PWA Web Push & Achievements	Build deduplicated in-app notifications, iOS/Android VAPID Web Push, and	Tables: notifications, push_subscriptions, user_badges. Modules: notifications,	GET /api/v1/notifications, POST /api/v1/push/subscribe, GET /api/v1/users/me/badges.	Verify notification rollup deduplication, iOS Home Screen install detection,	Disable outbound Web Push worker via Redis kill-switch while keeping in-app Activity Center

Phase	Objective & Prerequisites	Key Modules & Database Changes	Core API Contracts & Screens	Mandatory Tests & Completion Evidence	Rollback & Risk Mitigation
	initial Pip contribution badges. *Prereq: Phase 8.*	achievements.	Screens: SCR-10, SCR-31, SCR-32, SCR-35.	Service Worker offline /bookmarks caching, and maskable icon rendering.	live.
Phase 10: Trust, Safety, Block/Mute & Statutory Moderation	Harden block/mute filtering, ship statutory 24h/72h grievance reporting, moderator console, and account deletion purge. *Prereq: Phase 8, 9.*	Tables: user_blocks, user_mutes, safety_reports, audit_events. Module: safety.	POST /api/v1/safety/reports, POST /api/v1/safety/blocks, POST /api/v1/account/schedule-deletion. Screens: SCR-33, SCR-34, SCR-36 to SCR-39.	Verify bidirectional invisibility upon block across feed/search/chat; verify <60s ticket acknowledgment and complete R2 media deletion upon account purge.	Immediately lock public registration (INVITE_ONLY=true) if coordinated bot attack exceeds moderator queue capacity.
Phase 11: Performance Profiling, Load Testing & Launch Audit	Execute k6 load testing, Lighthouse CI, backup restore drill, and security audit. *Prereq: Phase 10.*	PgBouncer tuning, CDN cache rules, CSP headers.	All MVP endpoints under synthetic load.	Pass all 5 Release Gate tags (AUTOMATED-TESTED through SECURITY-REVIEWED).	Hold public launch until all p95 latency and security gates pass.
Phase 12: Post-Launch Enhancements (Short Video, Sprouts, 216 Poses)	Add Cloudflare Stream 60s video, 24h status Sprouts, and expand Pip library from 30 to 216 poses. *Prereq: Post-MVP validation.*	Cloudflare Stream webhook handler, sprouts table, full 216-pose SVG/Lottie manifest.	POST /api/v1/media/video-i[span_88](start_span)[span_88](end_span)ntents.	End-to-end adaptive HLS playback test and CPU/memory profiling on mobile.	Feature-flagged per cohort (video_uploads_enabled).

Autonomous Coding Agent Governance and Release Gates

To ensure AI coding assistants build reliable production software rather than fragile prototypes, repository execution is governed by strict rules enforced in AGENTS.md and CI pipelines. No coding agent may alter an approved product requirement, weaken an authentication or object-level authorization boundary, remove the Alembic startup preflight check, or substitute

mock fixtures for real database persistence without halting and recording an explicit justification in docs/decision-register.md for human owner approval. Every phase deliverable must attach verifiable evidence across all five release gates—migration up/down logs (AUTOMATED-TESTED), Testcontainers integration test output (INTEGRATION-VERIFIED), Playwright desktop and mobile screenshots matched against Stitch screen IDs (BROWSER-VERIFIED), k6/Lighthouse metrics (PERFORMANCE-VERIFIED), and authorization/upload security checks (SECURITY-REVIEWED)—before advancing to the next phase.

Document I: Master Decision Register and Final Architecture Audit

Architectural and Product Decision Register

ID	Decision Topic	Recommended Choice	Alternatives Considered	Evidence & Analytical Rationale	Cost & Complexity	Key Risks & Mitigations	Confidence	Revisit Condition	Owner Sign-Off
DEC-01	Central Product Thesis & MVP Boundary	Co-active social platform ("Co-Plots" + "Radish Patches" + Real-time Chat)	1. Pure TikTok/Reels clone 2. Pure Discord clone 3. Kitchen-sink super-app	Solves the real gap between passive media scrolling and fragmented group chat coordination while keeping MVP buildable.	Moderate complexity; $49–69/mo at launch.	Risk: Cold-start empty plots. Mitigation: Onboarding joins 3 active starter Patches (SCR-09).	High	If D30 Co-Plot participation falls below 10% in initial beta cohort.	Required
DEC-02	Launch Age Requirement	Strict 18+ Age Gate at MVP Launch	Allow 13–17 minors with parental consent flow	India's DPDP Act Section 9 mandates verifiable parental	Low complexity ($0 extra cost).	Risk: Excludes high-school demographic initially. Mitigation: Focus	High	When compliance budget justifies integrating DigiLocker/verifia	Required

ID	Decision Topic	Recommended Choice	Alternatives Considered	Evidence & Analytical Rationale	Cost & Complexity	Key Risks & Mitigations	Confidence	Revisit Condition	Owner Sign-Off
				consent for users under 18. Enforcing 18+ avoids heavy legal/KYC friction at MVP.		on university (18+) & hobbyist cohorts.		ble parental KYC.	
DEC-03	Visual Identity & Design Workflow	Dark-First "Nocturnal Botanical Studio" + DESIGN.md + Google Stitch MCP	1. Light-only serif theme 2. Pure black neon glassmorphism 3. Ad-hoc coding without Stitch	Stitch MCP (stitch.googleapis.com/mcp) + DESIGN.md enforces cohesive token discipline across 40 screens.	Free tier (350 Stitch generations/mo).	Risk: Raw Stitch HTML lacks ARIA state. Mitigation: Refactor into Radix/shadcn components.	High	If Stitch MCP quota is exceeded, upgrade to paid tier in Q4 2026.	Approved
DEC-04	Mascot Asset Architecture	Hybrid Layered SVG (30 core MVP \rightarrow 216 matrix) + 12 DotLottie animations	1. 200 static PNGs 2. 200 full Lottie files 3. No mascot	Layered SVGs weigh <8\text{KB}, adapt to light/dark themes, and prevent the 200-pose ambition from	Low runtime cost; zero layout shift.	Risk: Visual inconsistency across poses. Mitigation: Modular vector rig + CI SVG linter.	High	Expand from 30 core poses to full 216 matrix in Phase 12 after MVP launch.	Approved

ID	Decision Topic	Recommended Choice	Alternatives Considered	Evidence & Analytical Rationale	Cost & Complexity	Key Risks & Mitigations	Confidence	Revisit Condition	Owner Sign-Off
				blocking core engineering.					
DEC-05	Backend & Realtime Stack	Python FastAPI 0.137+ Modular Monolith + Native WebSockets + Redis Pub/Sub	1. Next.js Serverless only 2. Microservices + Kafka 3. Ably/Pusher SaaS	Next.js serverless routes drop persistent WebSockets; FastAPI + Redis Pub/Sub delivers low-cost, low-latency stateful sockets.	Low ($14–85/mo compute).	Risk: Dropped sockets during deploy. Mitigation: Client exponential backoff + sync.resume cursor gap fill.	High	If concurrent WebSocket connections exceed 50,000, evaluate dedicated Go/Elixir gateway or Ably.	Approved
DEC-06	Auth Verification Mechanism	Supabase Auth + Local PyJWT/JWKS Verification in FastAPI + Redis Denylist	1. Synchronous supabase.auth.get_user() on every request 2. Custom password auth	Local JWT signature verification runs in <0.5\text{ms} vs 80\text{ms}+ for remote HTTP verification while preserving immediate session revocatio	$0–25/mo up to 100k MAU.	Risk: Stale permissions during 15m access token TTL. Mitigation: Check Redis revocation set on sensitive mutations.	High	If MAU exceeds 500k, evaluate self-hosted GoTrue or native FastAPI OAuth.	Approved

ID	Decision Topic	Recommended Choice	Alternatives Considered	Evidence & Analytical Rationale	Cost & Complexity	Key Risks & Mitigations	Confidence	Revisit Condition	Owner Sign-Off
				n via Redis.					
DEC-07	Media Object Storage & Video	Direct-to-Cloudflare R2 Presigned Uploads + libvips (Phase 12 Video: Cloudflare Stream)	1. AWS S3 + CloudFront 2. API-proxied uploads 3. Self-hosted FFmpeg	Cloudflare R2's $0.00 egress saves 60–80%+ over S3; server-side HeadObject + libvips guarantees security and strips EXIF GPS data.	$0.08/mo (seed) to $245/mo (250k MAU).	Risk: Unattached orphan uploads. Mitigation: 6-hour arq cleanup cron on PENDING_UPLOAD rows.	High	Never for public egress; add Cloudflare Stream in Phase 12 for short video.	Approved
DEC-08	Search Infrastructure	PostgreSQL 16 Native tsvector (STORED) + pg_trgm GIN Indexes	1. Elasticsearch / OpenSearch 2. Algolia SaaS	Provides weighted full-text + typo-tolerant fuzzy search in a single database with zero sync lag or cluster overhead.	$0 extra infrastructure cost.	Risk: GIN index write overhead at massive scale. Mitigation: Restrict indexed fields to titles, handles, tags, and post bodies.	High	When posts table exceeds 15M rows or search p95 exceeds 100\text{ms}.	Approved

ID	Decision Topic	Recommended Choice	Alternatives Considered	Evidence & Analytical Rationale	Cost & Complexity	Key Risks & Mitigations	Confidence	Revisit Condition	Owner Sign-Off
DEC-09	Background Task Queue	arq (Asyncio Redis Queue)	1. FastAPI BackgroundTasks 2. Celery + RabbitMQ	Native asyncio execution shares FastAPI's async DB/HTTP code and reuses Redis with durable retries and timeouts.	$0 extra broker cost (shares Redis).	Risk: CPU-bound image resize blocking event loop. Mitigation: Run pyvips inside asyncio.to_thread() in worker pool.	High	If multi-day workflow orchestration is needed, evaluate Temporal.	Approved
DEC-10	End-to-End Encryption (E2EE) & Creator Payments	Explicitly Deferred / Excluded from MVP (TLS in transit + encryption at rest + private R2 buckets used instead)	Full Signal-protocol E2EE DMs and Stripe/Razorpay creator subscriptions in MVP	E2EE breaks web-first multi-device history, full-text message search, and statutory abuse moderation; payments introduce heavy tax/KYC/fraud overhead before PMF.	Saves 6–8 weeks of engineering and legal compliance cost.	Risk: Privacy-maximalist users may request E2EE. Mitigation: Be transparent in privacy docs that DMs are access-controlled, not E2EE.	High	Re-evaluate optional "Locked Vault" E2EE 1:1 chats and creator ticketing only after 50k+ DAU.	Required

Consistency Audit and Executive Launch Summary

A comprehensive cross-document verification confirms full alignment across all nine engineering documents. The co-active product thesis directly governs the Tier 1 and Tier 2 feature priorities, ensuring that rich media carousels, Co-Plot RSVP and slide-contribution blocks, Radish Patches, and real-time WebSocket threads operate as a unified loop. Every one of the 40 canonical screens maps to explicit FastAPI endpoints, normalized PostgreSQL 16 tables, two-plane authorization rules, and deterministic state machines—including explicit cleanup of file inputs and object URLs in SCR-15 after upload completion so no asset remains stuck in an unresolved state. Furthermore, India's IT Rules 2021 statutory SLA clocks (24h acknowledgment, 24h critical takedown, 72h law enforcement response, 15d resolution, and 30d GAC appeal window) and DPDP Act 2023 Section 9 minor protection rules are enforced across both user interfaces and database schemas.

To conclude the blueprint, the eight executive launch determinations are synthesized below:

1. **Recommended Product Thesis:** Swiftradish is a co-active social platform where interest-driven micro-communities and friend groups move seamlessly from rich media discovery to real-time coordination, collaborative remixing, and shared digital or real-world activities within a single installable PWA.
2. **Exact Initial MVP Feature Set:** The initial release encompasses 18+ authentication (OAuth and Email OTP), session management, and customizable dark-first profiles (SCR-04–SCR-09, SCR-23–SCR-24); a hybrid asymmetric follow and auto-detected mutual social graph with private account support (SCR-25); a multi-image carousel composer (up to 10 images) with direct-to-R2 presigned uploads, libvips EXIF stripping, alt-text, and deterministic post-upload reset (SCR-15–SCR-16); interactive **Co-Plot** cards supporting 1-tap RSVPs, collaborative photo carousel contributions, and auto-provisioned group chat threads (SCR-13, SCR-17–SCR-18); a dual-mode home feed pairing strict chronological **"My Patch"** with transparent, explainable **"Fresh Harvest"** discovery (SCR-11–SCR-12); contextual radish reactions, 3-level nested comments, quote remixes, and private bookmark collections (SCR-13, SCR-26); topic communities (**"Radish Patches"**) and unified PostgreSQL tsvector + pg_trgm full-text and fuzzy search (SCR-19–SCR-22); real-time 1:1, group, and Co-Plot messaging over WebSockets with client_msg_id idempotency, cursor gap sync, and a Message Requests quarantine inbox (SCR-27–SCR-30); a deduplicated in-app Activity Center, iOS/Android PWA Web Push notifications, and the 30-pose core **Pip the Radish** mascot and milestone badge system (SCR-10, SCR-31–SCR-32); and a complete Trust and Safety foundation with atomic blocking/muting, statutory 24h/72h grievance reporting, a moderator triage console, and 14-day account and media purge (SCR-33–SCR-40).
3. **Selected Architecture and Rationale:** The stack pairs a Next.js App Router PWA frontend with a Python 3.12+ / FastAPI 0.137+ modular monolith backend, PostgreSQL 16 (tsvector/pg_trgm), Redis 7 (arq workers and WebSocket Pub/Sub), Supabase Auth (local JWKS verification), and Cloudflare R2 with Cloudflare CDN. This architecture delivers sub-100ms API latency, stateful WebSockets that serverless functions cannot sustain, native full-text/fuzzy search without Elasticsearch overhead, and $0 media egress at an initial run-rate of $49 to $69 per month.
4. **First 10 Highest-Priority Decisions:** Codified in DEC-01 through DEC-10 in the Decision Register above, with explicit owner sign-off required on DEC-01 (MVP boundary), DEC-02 (18+ age gate), and DEC-10 (deferral of E2EE and creator payments).
5. **Largest Risks and Mitigations:** Cold-start empty Co-Plots are mitigated by requiring new users to join at least three active starter Patches during onboarding (SCR-09) and

launching within dense university and hobbyist cohorts. Schema drift between test suites and development/production databases is eliminated by banning Base.metadata.create_all() and enforcing the FastAPI lifespan Alembic revision preflight check. iOS PWA Web Push friction is mitigated by detecting non-standalone Safari sessions and presenting a guided "Add to Home Screen" drawer (SCR-10) only after a user joins their first Co-Plot or receives a direct message.

6. **First Implementation Milestone (Milestone M1 — Phases 1 to 4) and Acceptance Tests:** Milestone M1 delivers the approved DESIGN.md token file, the four anchor Stitch screens (SCR-11, SCR-13, SCR-15, SCR-28), the monorepo with Docker Compose and the Alembic preflight gate, working 18+ Supabase Auth with local FastAPI JWKS verification, profile customization, and the end-to-end direct-to-R2 multi-image upload and libvips processing pipeline. M1 passes only when: (a) FastAPI refuses to boot if the database Alembic revision mismatches HEAD; (b) <18 DOB registration returns HTTP 422; (c) invalid or expired JWTs return HTTP 401 in under 2\text{ms} locally; and (d) uploading three images streams directly to R2, strips EXIF GPS data, transitions to READY, and resets the composer cleanly to COMPOSER_EMPTY upon completion.
7. **Features Explicitly Excluded from Initial Release:** Short-form and long-form video transcoding (deferred to Phase 12 via Cloudflare Stream), 24-hour ephemeral status stories ("Sprouts"), the remaining 186 mascot pose expansions, WebRTC voice and video calling, end-to-end encryption (E2EE), and creator monetization or paid ticketing ledgers.
8. **Validation Experiments Required Before Committing to Deferred Ideas:** Before building 24-hour ephemeral "Sprouts," the team will measure whether at least 25% of created Co-Plots in closed beta receive two or more mutual contributions. Before enabling the paid Cloudflare Stream pipeline in Phase 12, the team will instrument the composer to measure how frequently beta users attempt to attach .mp4/.mov files or share external video links during active Co-Plots.

Works cited

1. Navigating Children's Privacy and Parental Consent Under The, https://thequantumhub.com/navigating-childrens-privacy-and-parental-consent-under-the-dpdp-act-2023/ 2. Indian perspective on protecting children's personal data | India, https://law.asia/childrens-data-protection-dpdp-act/ 3. Section 9: Processing of children's data - DPDP Act 2023, https://dpdpactindia.in/dpdp-act/section-9/ 4. Children's Data Under DPDP Act: Parental Consent Rules, https://ksandk.com/data-protection-and-data-privacy/child-data-protection-under-dpdp-act-parental-consent-rules/ 5. Your Users Are Misspelling Things and Postgres Can Handle It, https://tech-talk.the-experts.nl/postgres-fuzzy-search-trigrams-full-text-search-9e005f52deda 6. The pg_trgm extension - Neon Docs, https://neon.com/docs/extensions/pg_trgm 7. F.35. pg_trgm — support for similarity of text using trigram matching, https://www.postgresql.org/docs/current/pgtrgm.html 8. Sending web push notifications in web apps and browsers, https://developer.apple.com/documentation/usernotifications/sending-web-push-notifications-in-web-apps-and-browsers 9. Implementing Web Push Messages in 2024–2026 - Dr. Lex' Site, https://www.dr-lex.be/info-stuff/web-push.html 10. How to use Next.js as a backend for your frontend, https://nextjs.org/docs/app/guides/backend-for-frontend 11. WebSockets with Next.js: SSR, App Router, and Vercel, https://websocket.org/guides/frameworks/nextjs/ 12. FastAPI + HTMX: The No-Build Full-Stack - Blake Crosley, https://blakecrosley.com/guides/fastapi-htmx

13. Frontend Development Services - React, Next.js and Bootstrap, https://techypanther.com/services/frontend-development/ 14. Preparing Your App for Production: Icons, Metadata, and Beyond, https://juno.build/blog/preparing-your-app-for-production-icons-metadata-and-beyond 15. Implementing Supabase Authentication with Next.js and FastAPI, https://medium.com/@ojasskapre/implementing-supabase-authentication-with-next-js-and-fastapi-5656881f449b 16. Comprehensive Next.js Full Stack App Architecture Guide | Arno, https://arno.surfacew.com/posts/nextjs-architecture 17. Rights of Users* of IT Intermediaries - Grievance Appellate Committee, https://gac.gov.in/CMSData/CMSContent?qs=3fyhxH9mXj1t6SpSu/sU/w== 18. Overview of key regulations of IT Rules, 2021 - iPleaders, https://blog.ipleaders.in/overview-key-regulations-ir-rules-2021/ 19. Google Stitch MCP: From Design to Code with Claude Code in 23, https://pasqualepillitteri.it/en/news/647/google-stitch-mcp-export-claude-code-design-to-code 20. Duolingo Design System, Tokens and DESIGN.md | DesignMD, https://www.designmd.co/d/duolingo 21. duolingo's Design System: design guidelines, https://adele.uxpin.com/duolingo-design-guidelines 22. Duolingo Design System Elements - Jack Morgan - Dribbble, https://dribbble.com/shots/3836227-Duolingo-Design-System-Elements 23. Google Stitch MCP: 5-Min Setup + Is It Worth It (2026), https://justinmckelvey.com/blog/google-stitch-mcp 24. Stitch MCP - AIBase, https://mcp.aibase.com/server/1639703007623127440 25. MCP App for Google Stitch - Design Agent with interactive ... - GitHub, https://github.com/jayeshvpatil/stitch-mcp-app 26. Design-to-Code with Antigravity and Stitch MCP - Codelabs, https://codelabs.developers.google.com/design-to-code-with-antigravity-stitch 27. Stitch - Design with AI, https://stitch.withgoogle.com/ 28. PWA Push Notifications on iOS in 2026: What Actually Works, https://webscraft.org/blog/pwa-pushspovischennya-na-ios-u-2026-scho-realno-pratsyuye?lang=en 29. Video Streaming App Development Cost: 2026 CTO Guide - Fora Soft, https://www.forasoft.com/blog/article/video-streaming-app-development-cost-how-much 30. Cloudflare R2 vs S3 vs Backblaze B2 (2026) - DevOpsBoys, https://devopsboys.com/blog/cloudflare-r2-vs-aws-s3-vs-backblaze-b2-2026 31. Cloudflare R2 Pricing Explained - Real Costs vs S3 and Backblaze, https://mecanik.dev/en/posts/cloudflare-r2-pricing-explained-real-costs-vs-s3-and-backblaze/ 32. ARQ vs Celery, How to Run FastAPI Background Tasks with ... - Bithost, https://www.bithost.in/blog/tech-3/how-to-run-fastapi-background-tasks-arq-vs-celery-11 33. Managing Background Tasks in FastAPI: BackgroundTasks vs ARQ, https://davidmuraya.com/blog/fastapi-background-tasks-arq-vs-built-in/ 34. Celery Versus ARQ Choosing the Right Task Queue for Python, https://leapcell.io/blog/celery-versus-arq-choosing-the-right-task-queue-for-python-applications 35. Object Storage Pricing 2026: S3 vs R2 vs B2 vs Wasabi - Cloudzat, https://cloudzat.com/object-storage/ 36. R2 vs Backblaze B2: A Detailed Comparison for System Design, https://algoroq.io/compare-tech/r2-vs-backblaze-b2/ 37. Building a Supabase and FastAPI Project: A Modern Backend Stack, https://medium.com/@abhik12295/building-a-supabase-and-fastapi-project-a-modern-backend-stack-52030ca54ddf 38. PWAs on iPhone: What Works and What Does Not (October 2026), https://www.wavxsolutions.in/blog/pwa-on-ios-what-works 39. Do Progressive Web Apps Work on iOS? The Complete Guide for, https://www.mobiloud.com/blog/progressive-web-apps-ios/ 40. PWA on iOS: Install Guide & Limits for Advertisers 2026 - DeepClick,

https://deepclick.com/resources/blog/progressive-web-apps-on-ios/ 41. Web App Manifest Favicon Guide: Master Mobile Icons in 2026, https://favicondl.com/blog/web-app-manifest-favicon.html 42. Information Technology Rules, 2021 - Drishti IAS, https://www.drishtiias.com/daily-updates/daily-news-analysis/information-technology-rules-2021-2 43. Explained | The amendments to the IT Rules, 2021 - The Hindu, https://www.thehindu.com/sci-tech/technology/explained-the-amendments-to-the-it-rules-2021/article66079214.ece 44. Have you complained to a social media platform or requested, https://www.facebook.com/CyberDostI4C/posts/have-you-complained-to-a-social-media-platform-or-requested-content-removal-but-/1084241593995030/ 45. Summary of amendments to the Information Technology ... - Ikigai Law, https://www.ikigailaw.com/article/34/summary-of-amendments-to-the-information-technology-intermediary-guidelines-and-digital-media-ethics-code-rules-2021-in-relation-to-online-gaming 46. IT Rules, 2021 - A Step Towards Shaping Safer Social Media Platform, https://amlegals.com/it-rules-2021-a-step-towards-shaping-safer-social-media-platform/ 47. The best object storage for SaaS in 2026 - Botmonster Tech, https://botmonster.com/web-dev/the-best-object-storage-for-saas-in-2026/ 48. Web Application Manifest - W3C, https://www.w3.org/TR/appmanifest/ 49. Validating a Supabase JWT locally with Python and FastAPI, https://dev.to/zwx00/validating-a-supabase-jwt-locally-with-python-and-fastapi-59jf 50. Migrating from Static JWT Secrets to JWKS in Supabase - ObjectGraph, https://objectgraph.com/blog/migrating-supabase-jwt-jwks/